MLSys · im

First-Principles Infrastructure Modeling for Machine Learning Systems

Vijay Janapa Reddi
Harvard University

mlsysbook.ai/mlsysim

Abstract

An ML workload can fit in accelerator memory yet miss its latency target because memory bandwidth, communication, or queueing limits execution. Evaluating these constraints before acquiring hardware requires models whose assumptions can be inspected and changed. We present MLSys · im (Machine Learning Systems Infrastructure Modeling), a Python framework that combines established analytical models with typed workload, hardware, and infrastructure specifications. It organizes 22 bottlenecks into six domains and provides 276 composable models, solvers, and optimizers. Runtime units detect incompatible dimensions, while explicit efficiency inputs separate hardware ceilings from assumed sustained performance. Worked examples connect equations to code and expose the resource that limits each modeled configuration. The evaluation distinguishes comparisons with published performance, analytical lower bounds, accounting identities, and design-space search. These checks support reproducible first-order analysis; they do not establish a uniform prediction error across workloads or hardware. The framework serves the *Machine Learning Systems* textbook series and runs without an accelerator.

1 Introduction

Progress in machine learning continues to follow methods that exploit massive scale (Sutton, 2019). Dense frontier models now exceed 400 billion parameters (Llama Team, AI Meta, 2024) and sparse models pass a trillion (Fedus et al., 2022). Training these models requires dedicated datacenter infrastructure: pre-training Llama 3 405B, for example, harnessed up to 16,384 H100 GPUs (Llama Team, AI Meta, 2024) across network fabrics where memory capacity, bisection bandwidth, power delivery, and thermal limits cross-couple.

Yet the hardware required to develop intuition for these systems is scarce: a student cannot requisition thousands of GPUs to explore how network topology affects collective communication, and a researcher cannot easily sweep parallelism strategies across physical clusters. This leaves a widening gap between the scale of modern systems and the analytical tools available to reason about them.

Consider serving Llama 3 70B under a latency target. Its FP16 weights require about 141 GB, exceeding one H100's 80 GB capacity. Sharding the weights permits residency but introduces communication. Batching shares weight reads across requests while increasing KV-cache storage and compute work. Queueing adds a further delay under load. These mechanisms answer different questions: a configuration may fit but still miss its response-time target. We call a modeled resource limit a *wall*. A wall binds when the modeled configuration reaches that limit; more than one can bind, and the limiting resource can change after an input is perturbed. The framework reports these constraints separately rather than treating a single latency number as a deployment guarantee.

Modeling tools make different choices about fidelity, speed, and scope (Table 2). Profilers measure a concrete implementation on physical hardware. Network and architecture simulators resolve execution details that an analytical model omits. Analytical tools support faster exploration but differ in the workloads and resources they represent. MLSys · im combines several of these analytical views for education and early design decisions, with runtime units to help detect mistakes such as confusing work with throughput or bits with bytes. Its purpose is to make a calculation inspectable before a profiler or detailed simulator is used to test it.

MLSys · im (Machine Learning Systems Infrastructure Modeling) is an open-source, pure-Python, first-principles analytical modeling framework for ML systems, designed for education and early design-space reasoning before empirical benchmarking. It formalizes back-of-the-envelope ML systems reasoning into a dimensionally strict, modular engine. The framework cod-

Table 1: **The 22 ML Systems Walls.** Each wall represents a physical or logical constraint resolved by a dedicated resolver (model or solver). Walls 1–2 (Compute and Memory) share the `SingleNodeModel`; four companion models deepen individual walls (`TrainingMemoryModel` for Wall 2, `ServingCapacityModel` for Wall 4, `MoERoutingModel` for Wall 14, and `NetworkRooflineModel`, which lifts the Roofline to fleet scale). Together with 3 optimizers (parallelism, batching, placement), the framework provides 276 resolvers across 22 walls. Domains progress from local node resources through data movement and algorithmic scaling to fleet coordination, operations, and cross-cutting analysis. Each wall is formalized in Section 4.

#	Wall	Resolver	Bounded	Core Equation	Ref.
Node (Single-Accelerator Resources)					
1	Compute	SingleNode	Peak FLOP/s	$T = \text{OPs}/(\text{Peak} \times \eta)$	Williams et al. 2009
2	Memory	SingleNode	HBM BW + cap.	$T = \lvert W \rvert / BW_{\text{HBM}}$	Williams et al. 2009
3	Software	Efficiency	Silicon eff.	$\eta = \text{FLOPS}_{\text{ach}}/\text{Peak}$	Chowdhery et al. 2022
4	Serving	Serving	Prefill vs. dec.	$T_{\text{dec}} = \max(T_c, T_m) + T_{\text{layer}}$	Pope et al. 2023
5	Batching	Cont. Batch	KV allocation	$N_{\text{st}} = \lfloor M_{\text{KV}}/(kS_{\max}) \rfloor$; $N_{\text{pg}} = \lfloor M_{\text{KV}}/(kp\,\mathbb{E}\lceil S/p \rceil) \rfloor$	Kwon et al. 2023
6	Streaming	WeightStream	Injection BW	$T = \max(\lvert W_\ell \rvert / BW, 2P_\ell B/F\eta)$	Lie 2022
7	Tail Latency	TailLatency	P99 queueing	Erlang-C M/M/c	Harchol-Balter 2013
Data (Movement & Pipelines)					
8	Ingestion	Data	Storage I/O	$\rho = BW_{\text{demand}}/BW_{\text{supply}}$	Mohan et al. 2021
9	Transform.	Transform.	CPU preproc.	$T = B \cdot S_{\text{sample}}/C_{\text{cpu}}$	Murray et al. 2021
10	Locality	Topology	Bisection BW	$BW_{\text{eff}} = BW_{\text{link}} \cdot \beta/\text{osub}$	Leiserson 1985
Algorithm (Scaling & Compression)					
11	Complexity	Scaling	Scaling laws	$C = 6PD$; $P^* = \sqrt{C/120}$	Hoffmann et al. 2022
12	Reasoning	Inf. Scaling	Inf.-time comp.	$T = K \times T_{\text{step}}$	Snell et al. 2025
13	Fidelity	Compression	Acc.–efficiency	$r = b_{\text{base}}/b$; $r = 1/(1-s)$	Han et al. 2016
Fleet (Multi-Node Coordination)					
14	Communic.	Distributed	AllReduce	$T = 2\frac{N-1}{N}\frac{M}{BW_{\text{link}}} + 2(N-1)\alpha$	Patarasuk and Yuan 2009
15	Fragility	Reliability	Cluster MTBF	$\text{MTBF}_{\text{cl}} = \text{MTBF}_{\text{node}}/N$	Daly 2006
16	Multi-tenant	Orchestration	Queue wait	$T_{\text{wait}} = \rho/[2\mu(1-\rho)]$	Harchol-Balter 2013
Operations (Economics, Sustainability & Safety)					
17	Capital	Economics	TCO	$\text{TCO} = \text{CapEx} + \text{OpEx}$	Barroso et al. 2019
18	Sustain.	Sustainability	Carbon + water	$\text{CO}_2 = E \times \text{PUE} \times \text{CI}$	Patterson et al. 2021
19	Checkpoint	Checkpoint	I/O burst penalty	$\text{penalty} = T_{\text{write}}/T_{\text{interval}}$	Eisenman et al. 2022
20	Safety	Resp. Eng.	DP-SGD overhead	$\sigma \propto 1/\varepsilon$	Abadi et al. 2016
Analysis (Cross-Cutting Diagnostics)					
21	Sensitivity	Sensitivity	Binding constr.	$\partial T/\partial x_i$	Williams et al. 2009
22	Synthesis	Synthesis	Inverse spec	$BW_{\text{req}} = \lvert W \rvert / T_{\text{target}}$	Williams et al. 2009
Optimizers (Design-Space Search, no dedicated wall)					
	Parallelism	Parallelism	Max MFU	$\max_c \text{MFU}(c)$ over TP×PP×DP	Shoeybi et al. 2019
	Batching	Batching	SLA-aware B^*	max B s.t. $P_{99} \leq T_{\text{SLA}}$	—
	Placement	Placement	Cost–perf.	min cost s.t. $T \leq T_{\text{SLA}}$	—

ifies 22 "Systems Walls" into 276 composable resolvers organized across six domains: Node, Data, Algorithm, Fleet, Operations, and Analysis. It separates computational *demand* from hardware *supply* and environmental *context* through a 5-layer demand–supply architecture, enforces unit correctness at runtime via the `pint` library, and supports interactive analysis on a laptop; Section 6.2 reports the runtime of a specified sweep.

Each resolver combines equations with hardware specifications and declared modeling inputs. For compute time, the efficiency coefficient η scales peak FLOP/s to an assumed sustained rate. It can represent effects such as tile utilization and non-matrix operations, but

the solver does not derive those effects from a kernel trace. Communication, memory traffic, and pipeline overhead appear as separate terms where the selected solver models them. This separation makes assumptions visible and permits sensitivity analysis; it does not make an assumed efficiency an independently predicted quantity (Williams et al., 2009).

The textbook series is the framework's primary consumer (Janapa Reddi, 2025). Students can change a batch size or bandwidth and follow the resulting change through an equation, a solver, and a plotted result. Instructors can use the same calculations in classroom demonstrations, and engineers can compare candidate configurations before measuring them. The package is open source under Apache-2.0 and runs without an accelerator. Committed scripts regenerate the paper's solver outputs and data figures; published reference values retain their source attribution. Appendices A and B describe textbook integration and the registry surface. The evaluation reports individual comparisons and their limitations rather than a framework-wide accuracy bound.

MLSYS·IM does not introduce new physical laws or mathematical bounds. The individual equations come directly from prior papers and textbooks (cited across Table 1). Instead, our contribution is bringing these scattered models together into a single, executable package that enforces physical units, chains them to find the binding bottleneck, and lets students and engineers run the math on a laptop.

This paper makes the following contributions:

- **C1. A 22-Wall Taxonomy.** An organization of 22 common bottlenecks across six domains into a single taxonomy that connects each constraint directly to its published equation (Section 4).
- **C2. Demand–Supply Separation and Unit Safety.** A 5-layer software architecture that separates what a model computes from where it runs, backed by runtime unit checking that catches silent calculation errors (Section 3).
- **C3. Composable Solvers and Hardware Synthesis.** A collection of 276 resolvers that chain together to evaluate feasibility, performance, and fleet cost, plus an inverse solver that calculates the minimum hardware needed to meet a latency target (Sections 5 and 7).
- **C4. An Open Companion for Systems Education.** A lightweight, reproducible tool that supports the *Machine Learning Systems* textbook (Janapa Reddi, 2025), allowing students to explore cluster-scale trade-offs without requiring access to expensive hardware (Section 7).

The following sections connect the software architecture to the equations it implements, then examine the evidence supporting its outputs. The case studies show how those outputs inform a design decision; the limitations identify where measurement or a more detailed model remains necessary.

2 Related Work

The tools below answer related questions at different levels of detail. Their published runtimes and error rates come from different workloads and evaluation protocols, so they are not a controlled ranking. Table 2 summarizes their roles; the text identifies representative sources.

2.1 Cycle-Level Simulators

gem5 (Binkert et al., 2011) supports detailed processor and memory-system simulation. That detail is useful when pipeline behavior or cache interactions determine the result. An analytical model instead begins from aggregate work and resource rates, which makes it easier to vary a hardware specification but omits the execution mechanisms that a detailed simulator can resolve.

ASTRA-sim 2.0 (Won et al., 2023) addresses the ML gap with a hierarchical system simulator purpose-built for distributed training. It models collective communication patterns such as AllReduce across realistic network topologies through pluggable network backends. Its cycle-level Garnet backend took 21.42 minutes to simulate a 1 MB AllReduce on 64 NPUs, while its analytical backend ran the same case in 1.70 seconds. SimAI (Wang et al., 2025) extends this approach with a full-stack training simulator that integrates NS3-based network modeling with kernel computation traces, reporting an average of 98.1% alignment with real-world results on clusters of up to 1,024 GPUs. These published timings illustrate the cost of resolving communication details. They do not establish a general runtime ratio against MLSysIM, whose workload and outputs differ.

MLSYS·IM occupies a different point on the fidelity–speed spectrum. Where ASTRA-sim answers "how many microseconds does this AllReduce take on this exact topology," MLSYS·IM answers "which of 22 possible bottlenecks binds this system, and how does changing hardware shift the binding constraint?" The two approaches are complementary: MLSYS·IM narrows the design space, and high-fidelity simulators validate specific points within it.

Table 2: **Roles of ML systems modeling tools.** Scope summarizes the cited publications rather than an exhaustive audit of current software. Phase and distributed support follow those descriptions. The table does not rank accuracy or runtime across unmatched workloads.

Tool	Approach	Scope	Phase	Dist.
Cycle-level				
gem5	Cycle-accurate	CPU/GPU microarchitecture	Both	–
ASTRA-sim 2.0	System sim.	Network collectives, topology	Train	✓
SimAI	Trace-driven	Full-stack distributed training	Train	✓
Accelerator design				
Timeloop + Accelergy	Analytical	Accelerator dataflow & energy	Infer	–
LLMCompass	Analytical	LLM inference HW design space	Infer	○
Analytical & co-design				
Paleo	Analytical	DNN training compute & comm.	Train	✓
Calculon	Analytical	LLM training performance	Train	✓
Lumos	Trace-driven	LLM training perf. modeling	Train	✓
Vidur	Empirical	LLM inference scheduling	Infer	–
GenZ	Analytical	LLM inference platform design	Infer	–
LLM-Viewer	Analytical	LLM inference memory/latency	Infer	–
Sustainability				
LLMCarbon	Analytical	LLM carbon footprint (op. + embodied)	Both	–
CodeCarbon	Empirical	Runtime energy & carbon tracking	Both	–
MLSys · IM	**Analytical**	**Full-stack: compute, memory, network, data, scaling, reliability, econ., sustainability, safety**	**Both**	✓

2.2 Accelerator Design Tools

A second class of tools targets the design and evaluation of individual accelerator architectures. Timeloop (Parashar et al., 2019) provides a systematic methodology for evaluating DNN accelerator dataflows, modeling how data tiles map onto spatial architectures and estimating latency and energy for each mapping. Accelergy (Wu et al., 2019), its companion framework, supplies the energy estimation primitives that Timeloop consumes. Together, they form a powerful toolkit for accelerator architects exploring the design space of novel silicon. LLMCompass (Zhang et al., 2024) brings this approach to LLM inference, combining an automated mapper with an area-based cost model to explore compute, memory bandwidth, and buffer configurations, achieving 4.1% error across prefill and decoding on a 4-GPU A100 node, with a GPT-3 evaluation taking 16 minutes.

These tools operate at the operator and tile level, modeling how a single convolution or matrix multiplication executes on a specific microarchitecture. They do not reason about fleet-level concerns such as how the data pipeline feeds a cluster or what the total cost of ownership looks like at scale. MLSys · IM operates one abstraction level higher. It consumes the *outputs* of accelerator-level analysis (peak FLOP/s, memory bandwidth, TDP) as inputs to its hardware registry and reasons about how those specifications interact with workload demands, network topologies, and infrastructure constraints.

2.3 Analytical and Co-Design Tools

Closest in spirit to MLSys · IM are analytical tools that sacrifice microarchitectural detail for speed. Calculon (Isaev et al., 2023) is an analytical co-design tool for large language model training. It models training time as a function of hardware specifications, parallelism strategies, and model architecture, achieving execution speeds comparable to MLSys · IM. Its published focus is transformer training performance and hardware co-design. Lumos (Liang et al., 2025) takes a trace-driven approach, using profiled kernel traces to predict LLM training performance at scale with 3.3% average error on up to 512 H100 GPUs. Those results rely on profiled traces. MLSysIM accepts aggregate hardware specifications, with less information about kernel execution; the two error figures are not directly comparable. The DeepSeek-V3 systems paper (DeepSeek-AI, 2025) exemplifies the kind of hardware-aware co-design analysis that MLSys · IM targets: it demonstrates how FP8 mixed-precision training, MoE sparsity, and multi-plane network topology interact to achieve frontier-model training at a fraction of conventional cost—a multi-wall

optimization that spans Walls 1, 3, 13, 14, and 17 in our taxonomy.

Paleo (Qi et al., 2017) pioneered the analytical approach, decomposing DNN training time into computation and communication components across data- and model-parallel configurations. Earlier analytical efforts such as FlexFlow (Jia et al., 2019) optimized parallelism strategies through simulation-guided search, and Habitat (Yu et al., 2021) provided cross-hardware extrapolation of training performance using execution-time scaling curves. While these tools advanced specific aspects of analytical modeling, they predated or did not address the full scope of modern concerns (inference serving, fleet economics, sustainability). Vidur (Agrawal et al., 2024) extends analytical modeling to LLM *inference*, using operator-level profiling to build a fine-grained runtime estimator validated at less than 5% error across multiple LLMs and scheduling policies. GenZ (Bambhaniya et al., 2024) provides an analytical framework for LLM inference platform design that models multi-dimensional network topologies and serving optimizations. DistServe (Zhong et al., 2024) and Sarathi-Serve (Agrawal et al., 2025) advance LLM serving through prefill-decode disaggregation and chunked-prefill scheduling, respectively, demonstrating that the two-phase inference model (Equation (7)) requires increasingly sophisticated scheduling to achieve high goodput. These works emphasize inference performance and scheduling. Their detailed models can test configurations suggested by a broader resource analysis.

LLM-Viewer (Yuan et al., 2024) and llm-analysis (Li, 2023) provide lightweight memory and latency estimation for transformer training and inference. These tools are useful for single-model profiling but do not extend to fleet-level reasoning, multi-tenant scheduling, or cross-domain constraint analysis.

A parallel line of work targets sustainability and fleet efficiency. LLMCarbon (Faiz et al., 2024) projects end-to-end carbon footprints (operational and embodied) for dense and MoE LLMs, reporting differences of at most 8.2% against published footprints. CodeCarbon (Courty et al., 2024) tracks the energy use and carbon emissions of code as it runs. Wongpanich et al. (2025) introduce *ML Productivity Goodput* (MPG) as a fleet-level efficiency metric for warehouse-scale TPU clusters, demonstrating that traditional utilization metrics are insufficient for characterizing ML fleet performance across model, data, framework, compiler, and scheduling layers. They provide complementary ways to estimate or measure energy and emissions; their system boundaries must be retained when comparing results.

MLSys·IM combines models from several of these domains behind a common Python interface. Its contribution is the composition and educational presentation of those models, including units and source annotations. Breadth does not imply greater fidelity within any one domain, and a common interface does not remove the need to reconcile assumptions between models.

2.4 Pedagogical Precedents

MLSys·IM draws direct inspiration from the tradition of pedagogical simulators in systems education. SPIM, James Larus's MIPS simulator used alongside Patterson and Hennessy's textbook (Patterson and Hennessy, 2014), taught generations of students computer architecture not by replicating a production processor, but by running programs on a simplified instruction-set model that made architectural concepts tangible through rapid experimentation. Similarly, xv6 (Cox et al., 2011) and MINIX (Tanenbaum and Woodhull, 2006) teach operating systems by stripping away production complexity to reveal core abstractions.

MLSys·IM follows this pedagogical philosophy for the ML systems domain. Production ML infrastructure (spanning millions of lines of code across frameworks, compilers, schedulers, and orchestrators) is too complex for students to reason about directly. MLSys·IM provides a controlled environment in which students can sweep hardware configurations, vary parallelism strategies, and observe how binding constraints shift, all in rapid iteration cycles. Its integration with the companion textbook (Janapa Reddi, 2025) supports structured laboratory exercises, which the research-oriented tools surveyed above do not target.

3 Architecture

A workload describes demand; hardware supplies compute and memory; infrastructure and topology determine the surrounding constraints. Figure 1 shows how these inputs reach the resolvers. The architecture keeps the input descriptions separate so that changing hardware does not require rewriting a workload, and changing a regional energy profile does not alter the performance model.

3.1 Design Philosophy

A change to one resource can expose another limit. Increasing compute throughput can leave memory traffic unchanged, while adding accelerators can increase communication time. The architecture separates these terms and retains their units so that a reader can trace the result back to its inputs. Four principles guide that organization.

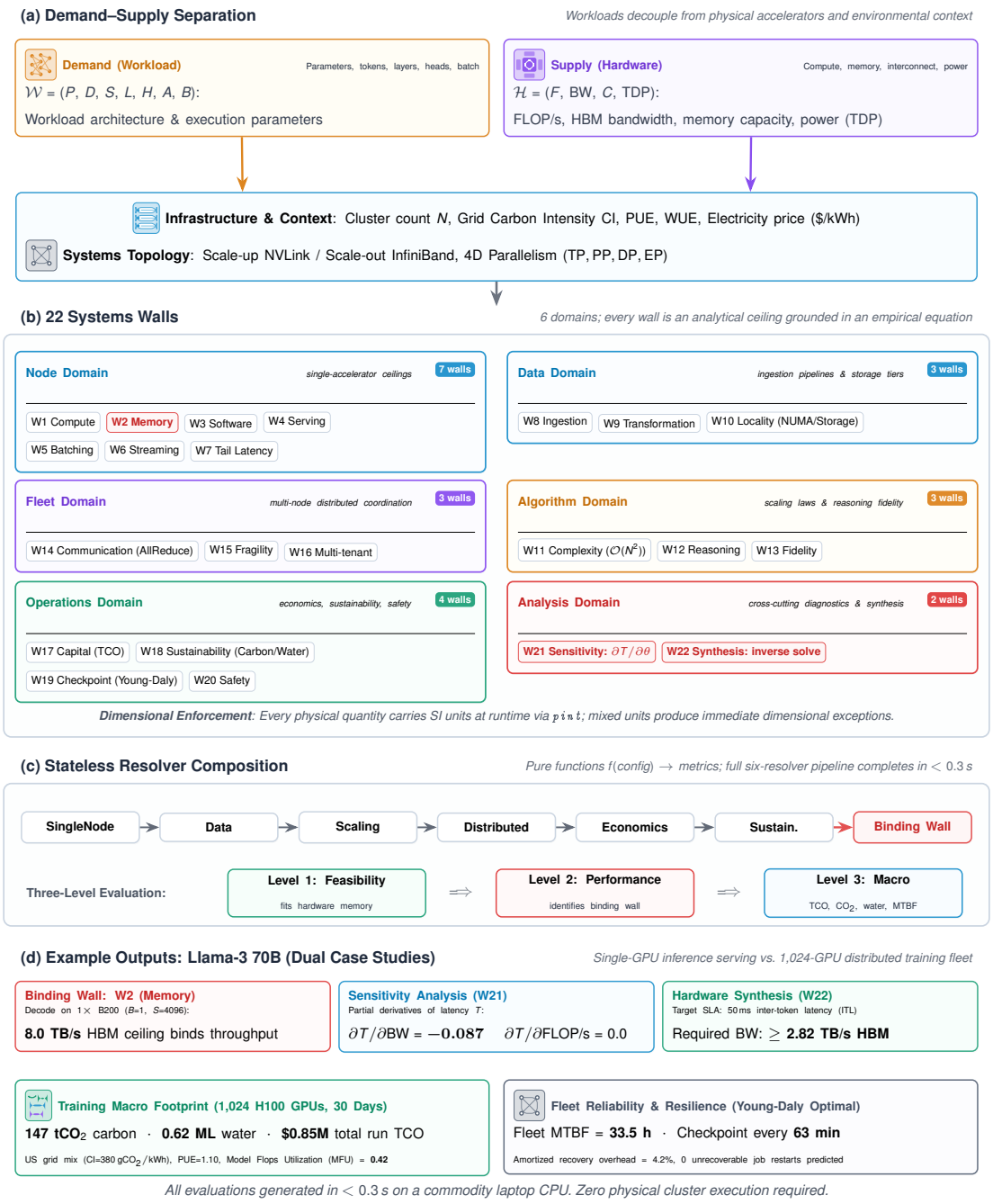


Figure 1: **MLSys · im Framework Overview.** (a) Demand–supply separation decouples workload specifications from hardware capabilities and environmental context. (b) All 22 Systems Walls organized into 6 domains (Node, Data, Algorithm, Fleet, Operations, Analysis), each grounded in a published equation. (c) Stateless resolver composition chains 276 resolvers to identify binding constraints through a three-level evaluation. (d) Example outputs for Llama-3 70B: decode on one B200 (binding wall, sensitivities, and synthesized bandwidth) and a 1,024-GPU H100 training fleet at MFU 0.42 (carbon, water, TCO, and reliability), all six solves completing in about 0.002 s on a laptop.

3.1.1 Analytical Speed over Cycle Accuracy

Analytical models enable rapid design-space exploration that cycle-accurate simulators cannot support. By parameterizing single-device compute throughput with an explicit silicon efficiency coefficient η_{silicon} (reflecting systolic tile quantization and non-GEMM vector unit overheads), MLSys · im evaluates configurations in sub-second time without requiring cycle-accurate microar-

chitectural simulation. This enables sweeps across thousands of hardware, model, and topology combinations in seconds—the kind of iterative "what-if" exploration that Hennessy et al. (2024) identify as essential for quantitative architectural reasoning.

To maintain rigor despite analytical simplification, MLSYS·IM enforces a "No Magic Numbers" invariant. Registry values carry a typed `Provenance` record whose *kind* (datasheet, literature, industry report, convention, estimate, derived, illustrative, or heuristic) determines what evidence it must supply: evidence-backed kinds require a source URL, estimates and derived values require explanatory notes, and all records require a verified date. The H100's FP16 rate is not a bare `989` floating-point literal; it is `989 * TFLOPs / second`, half of the 1,979 teraFLOPS FP16 Tensor Core figure that NVIDIA publishes *with sparsity* (NVIDIA Corporation, 2023). Shared sources live once in a central catalog of 222 provenance records (`core.provenance_catalog`) that registry entries reference by key, and an audit tool (`python -m mlsysim.tools.audit_provenance`) walks every registry and fails on any value with missing or weak provenance. These records make the inputs traceable. Their presence does not by itself verify the source or the model that uses the value.

3.1.2 Dimensional Strictness as an Invariant

Dimensional consistency is a pervasive challenge in systems modeling. Mixing gigabits with gigabytes and omitting the refractive index of fiber in latency calculations are canonical failure modes that silently corrupt results. MLSYS·IM treats dimensional correctness not as a feature but as a runtime invariant, within unit-aware arithmetic. The framework wraps every physical quantity using the `pint` unit library, and gives compute work its own base dimension: a FLOP is not a dimensionless count, so compute throughput can never silently add to, or convert into, a memory bandwidth. That confusion is the category error at the heart of many back-of-the-envelope mistakes. Mixing incompatible dimensions (such as compute throughput, bandwidth, or latency) raises a deterministic `DimensionalityError` when the incompatible operation is evaluated, while valid ratios still compose: arithmetic intensity (FLOP/byte) divides cleanly, and utilization (achieved over peak FLOP rate) reduces to a dimensionless fraction:

```python
from mlsysim.core.units import Q_
rate = Q_("989 TFLOPs/s")     # Compute throughput
bw   = Q_("3.35 TB/s")        # Memory bandwidth
(rate / bw).to("flop/byte")   # → 295 flop/byte (ridge point)
rate + bw  # → raises DimensionalityError
```

Listing 1: **Dimensional Strictness.** Prevents silent unit errors at the API level.

This design catches incompatible dimensions and performs explicit unit conversions. It cannot detect a missing term or a numerical value entered in the wrong unit before that unit is attached.[1] Hardware and model specifications live in typed registries (e.g., `Hardware.Cloud.H100.memory.bandwidth = 3.35 * TB/s`); the unit vocabulary lives in `core.units`, and physical constants and formulas live in `mlsysim.physics`, preventing unit mismatches at the API level. The physics formulas additionally validate the *dimensionality* of their arguments, raising `DimensionalityError` when, for example, a byte count is passed where a bandwidth is expected.

3.1.3 Taxonomic Coverage

We aim for broad coverage of the bottlenecks that have stable analytical formulations rather than for completeness, and Section 9 lists the constraints we deliberately exclude. MLSYS·IM codifies 22 such bottlenecks, which we call *Systems Walls*, organized into six domains: Node (single-accelerator resources), Data (movement and pipelines), Algorithm (scaling and compression), Fleet (multi-node coordination), Operations (economics, sustainability, and safety), and Analysis (cross-cutting diagnostics). Each wall maps to a resolver with a formal equation grounded in the systems literature. Section 4 presents the full taxonomy with formal definitions, and Section 5 describes how solvers compose.

3.1.4 Demand–Supply Separation

MLSYS·IM enforces a strict separation between *what* a model computes and *where* it runs. A `TransformerWorkload` describes computational demand (parameters, layers, FLOPs, arithmetic intensity) without reference to any specific accelerator. A `HardwareNode` describes physical supply (peak throughput, memory bandwidth, TDP) without reference to any specific model. This decoupling, inspired by the compiler IR philosophy of separating representation levels (Hennessy et al., 2024), enables hardware–software co-design. The same GPT-3 workload can be evaluated against an H100, a TPU v5p, or a hypothetical future accelerator in a single parametric sweep, with all dimensional conversions handled automatically.

[1] The most infamous unit-conversion failure is the Mars Climate Orbiter, lost in 1999 because ground software produced thrust data in pound-force seconds while the spacecraft expected newton seconds (Stephenson et al., 1999).

Table 3: **Supported Workload Types.** Each workload lowers to a `ComputationGraph` with total FLOPs, weight bytes, and arithmetic intensity.

Workload	Key Parameters	Scaling
Transformer	P, L, H, a, seq. length	$2P$ FLOPs/token
CNN	P, inference FLOPs	Fixed per image
Sparse (MoE)	Total vs. active P, experts	Active P for FLOPs
SSM (Mamba)	P, state dim, D	$O(1)$ state cache
Diffusion	P, denoising steps T	$T\times$ FLOPs/step

3.2 The Layered Input Stack

MLSYS·IM implements these design principles through a five-layer architecture (Figure 2). Layers A–D are *input layers* that describe demand, supply, context, and topology, respectively. Layers A–C are independent of one another, and Layer D composes Layer B accelerators and, optionally, a Layer C grid or datacenter into a fleet. The single lowering step occurs in Layer A, where a workload's `lower()` method produces a hardware-agnostic `ComputationGraph` intermediate representation. Layer E (Resolvers) then consumes any combination of layers A–D as needed; a single-node analysis requires only A+B, while a fleet-wide carbon estimate draws on A+B+C+D.

Layer A: Workloads (Demand). A workload is a hardware-agnostic description of computational demand. MLSYS·IM provides 6 concrete workload types (Table 3), each exposing a `lower()` method that produces a `ComputationGraph` (an intermediate representation containing total operations, weight bytes, and arithmetic intensity in `flop/byte`). This IR is the contract between demand and supply, capturing *what* must be computed without prescribing *how*.

Layer B: Hardware (Supply). A `HardwareNode` composes four subsystems: `ComputeCore` (peak FLOP/s at the base FP16 precision plus a precision-keyed dictionary, e.g., TF32, FP8, INT8, FP32), `MemoryHierarchy` (capacity and bandwidth), optional `StorageHierarchy`, and optional `IOInterconnect`. Each node also carries TDP, unit cost, and a kernel dispatch tax. The `ridge_point()` method computes the Roofline inflection $R = F_{\text{peak}}/BW_{\text{mem}}$ in `flop/byte` (Williams et al., 2009), enabling immediate classification of any lowered workload as compute-bound or memory-bound.

Layer C: Infrastructure (Context). `GridProfile` objects encode regional environmental parameters: carbon intensity (gCO_2/kWh), Power Usage Effectiveness (PUE), and Water Usage Effectiveness (WUE). A `Datacenter` composes a grid profile with an optional facility PUE override; rack power density is described separately by `RackProfile`. This layer converts raw energy consumption into carbon footprint and water usage, following the methodology of Patterson et al. (2021).

Layer D: Systems (Topology). A `Fleet` composes `Node` instances (each a physical compute server containing one or more accelerators on PCIe or NVLink) with a `NetworkFabric` (topology, inter-node bandwidth, latency, oversubscription ratio). This layer enables distributed analysis. The `DistributedModel` decomposes workloads using 4D parallelism (data-parallel DP $\times$ tensor-parallel TP $\times$ pipeline-parallel PP $\times$ expert-parallel EP) and calculates hierarchical AllReduce costs, pipeline bubble fractions, and scaling efficiency.

Layer E: Resolvers (Analysis). The 276 stateless resolvers (271 models, 2 solvers, and 3 optimizers) consume demand, supply, context, and topology to produce dimensioned performance metrics. The solver formalism (stateless composition, chaining semantics, and three-level evaluation) is detailed in Section 5.

The five layers are useful only if their inputs are reproducible. MLSYS·IM therefore organizes vetted specifications into 12 curated registries collectively called the *MLSys Zoo*. The 8 core registries cover Hardware (accelerators, boards, and technology classes under `Hardware.Tech`), Models (workload architectures across language, vision, generative vision, recommendation, state-space, and TinyML), Datasets (benchmark catalog entries), Platforms (deployment envelopes), Infrastructure (grids, datacenters, racks, cooling, pricing, and capacity), Systems (nodes, racks, fabrics, clusters, pods, and storage paths), Ops (operational policies and overhead profiles), and executable Scenarios, which compose these reusable entries with operational constraints (such as latency targets, power limits, or geographic regions) without redefining underlying facts. The extension registries, Agents (agentic workload profiles), Embodied (robot platforms), Sensors (perception sensors), and Actuators (robotic actuators and joint transmissions), sit outside the wall taxonomy; Section 9 explains why. Two auxiliary surfaces complete the picture: non-executable reference baselines, such as Waymo data-rate ranges or mobile power envelopes, live in `ReferenceStats.*`, and cited field figures (MFU bands, scaling laws) live in `Literature.*`. Solver fallbacks live in `mlsysim.engine.calibration`; architecture formulas (Roofline, AllReduce, Young–Daly) live in `mlsysim.physics`; the machine-readable wall taxonomy of Table 1 ships as `mlsysim.engine.walls`; and domain-reviewed reference envelopes in `mlsysim.engine.empirical`

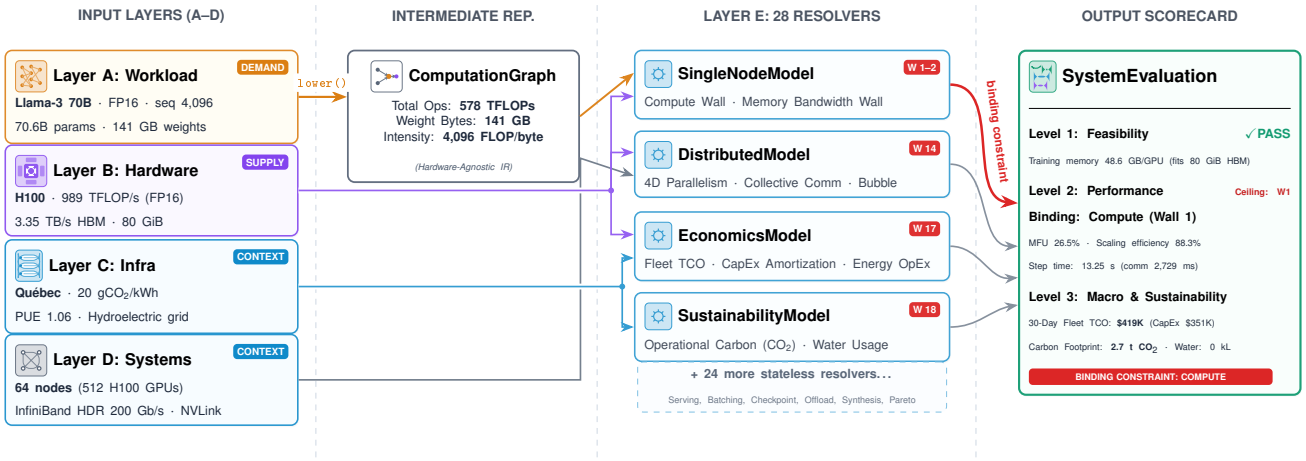


Figure 2: **The MLSys · im 5-Layer Architecture.** Layers A–D provide typed inputs: workload demand, hardware supply, infrastructure context, and network topology. The single lowering step occurs in Layer A, where `lower()` produces a hardware-agnostic Computation Graph (an intermediate representation of total FLOPs, weight bytes, and arithmetic intensity). Layer E's 276 stateless resolvers consume these inputs and produce a three-level SystemEvaluation scorecard. The red arrow marks the binding constraint identified by the solver chain.

bind registry entries to published benchmark envelopes for regression testing.

Registry entries are authored as YAML data files and validated against `pydantic` schemas at import time, so stating a fact is a data edit while computing or composing remains Python. The loader rejects duplicate keys outright and resolves two reference forms: `@tech:` pointers that let an instance inherit a technology-class fact, and `@prov:` pointers into the shared provenance catalog (Section 3.1.1). The `Hardware.Tech` sub-registry encodes the load-bearing distinction between *instances* and *technology classes*: per-part facts that vary between products (capacity, bandwidth, TDP, price) live on instances such as `Hardware.Cloud.H100`, while facts that are properties of a technology generation (access latency, energy per operation, interconnect latency) live once on tech-class entries. For example, `Hardware.Cloud.H100` takes its NVLink latency from `Hardware.Tech.Interconnect.NVLink` through an `@tech:` reference, and instance lookups fall back to the tech class when a value is unset. Each registry entry is a fully typed object; for example, `Hardware.Cloud.H100` returns a `HardwareNode` with all physical quantities dimensioned via `pint`. Figure 3 illustrates how these reusable facts become a concrete infrastructure model: a scenario binds a workload, accelerator fleet, regional grid, and operational policy to a local question, then passes the resulting typed objects to the resolver chain.

Once a scenario has assembled typed inputs, the `physics` package decomposes the framework's analytical equations into 27 domain-aligned modules (Table 4), each owning one conceptual domain of the systems stack, plus two supporting modules: `physics.constants` holds universal physical constants (e.g., the speed of light in fiber), and `physics.quantities` provides generic dimensioned helpers (transfer time, compute time, energy from power, carbon from energy). A shared `_ensure_unit()` helper coerces raw floats into `pint` quantities at module entry points, assigning the documented default unit to untyped inputs; a wrong unitless value can still pass. Chapter and notebook code import from `mlsysim.physics` directly. The appendix (Table 10) lists representative function signatures.

The Hardware registry holds 52 devices across five deployment tiers (Cloud, Workstation, Edge, Mobile, Tiny), spanning nine orders of magnitude in peak throughput from milliwatt-class microcontrollers (`Hardware.Tiny.ESP32_S3`, 520 KiB on-chip SRAM) to wafer-scale engines (`Hardware.Cloud.Cerebras_CS3`, 44 GB on-wafer SRAM, 125 PFLOP/s). Beyond NVIDIA GPUs (from T4 and V100 through B200 and GB200 NVL72), the registry includes AMD MI250X and MI300X, Intel Gaudi 2 and Gaudi 3, AWS Trainium 2, and Google TPU generations v1 through v6/Trillium. For heterogeneous and constrained deployments, it also models workstations (Apple M3 Max), edge boards (NVIDIA Jetson, Google

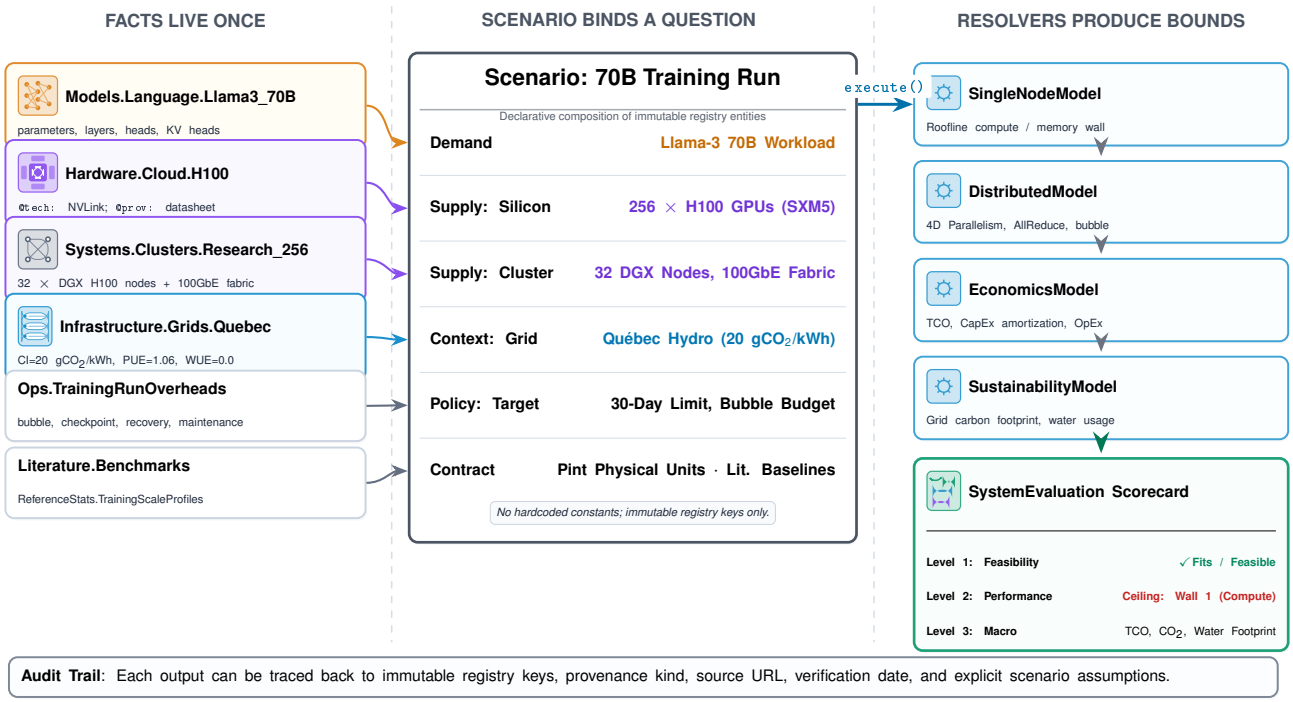


Figure 3: **Registry-Backed System Composition.** MLSYS·IM stores workload, hardware, systems, infrastructure, operational, and literature facts once, with typed schemas and provenance records. A scenario composes those reusable facts into a local infrastructure question—for example, training a 70B model on a 256-GPU H100 fleet in Québec or Iowa—and hands dimensioned objects to the resolver chain. The resulting `SystemEvaluation` can be traced back to the registry keys, source records, and scenario assumptions that produced it.

Coral), mobile SoCs (Apple A17 Pro, Snapdragon 8 Gen 3, Google Tensor G3), and TinyML microcontrollers (ESP32-S3, nRF52840). The `list(sort_by=)` class method enables programmatic comparison, and users extend any zoo by instantiating new typed objects (Listing 2) or adding YAML data files, ensuring custom entries participate in the same dimensionally strict pipeline as vetted ones. Figure 4 maps the implementation structure behind this composition path: panel (a) shows package architecture (registries, physics modules, core utilities, and invariant checks), while panel (b) traces a concrete analysis from typed inputs through the resolver chain to the three-level `SystemEvaluation` scorecard. Section B gives a representative snapshot of the current registry surface.

3.3 The Type System

MLSYS·IM's type system is built on Pydantic `BaseModel` classes with `pint Quantity` fields, providing both schema validation and dimensional enforcement at construction time. The same schemas validate the YAML registry data: every `*.yaml` entry is instantiated through its `pydantic` type at import, so a malformed unit string fails the import rather than surfacing later as a wrong number, and a missing provenance record fails the provenance audit. The composition hierarchy is deliberately shallow: `HardwareNode` aggregates `ComputeCore` and `MemoryHierarchy` as direct fields, not through deep inheritance. This design makes the relationship between a hardware specification and its physical quantities immediately legible:

```python
from mlsysim.hardware.types import *
from mlsysim.core.units import Q_
node = HardwareNode(
    name="Custom Accelerator",
    release_year=2025,
    compute=ComputeCore(
        peak_flops=Q_("500 TFLOPs/s"),
        precision_flops={"fp8": Q_("1000 TFLOPs/s")
    }),
    memory=MemoryHierarchy(
        capacity=Q_("96 GiB"),
        bandwidth=Q_("4 TB/s")),
    tdp=Q_("500 W"))
print(node.ridge_point())  # → 125.0 flop/B
```

Listing 2: **Custom Hardware Node.** Composing a hardware specification with dimensional types.

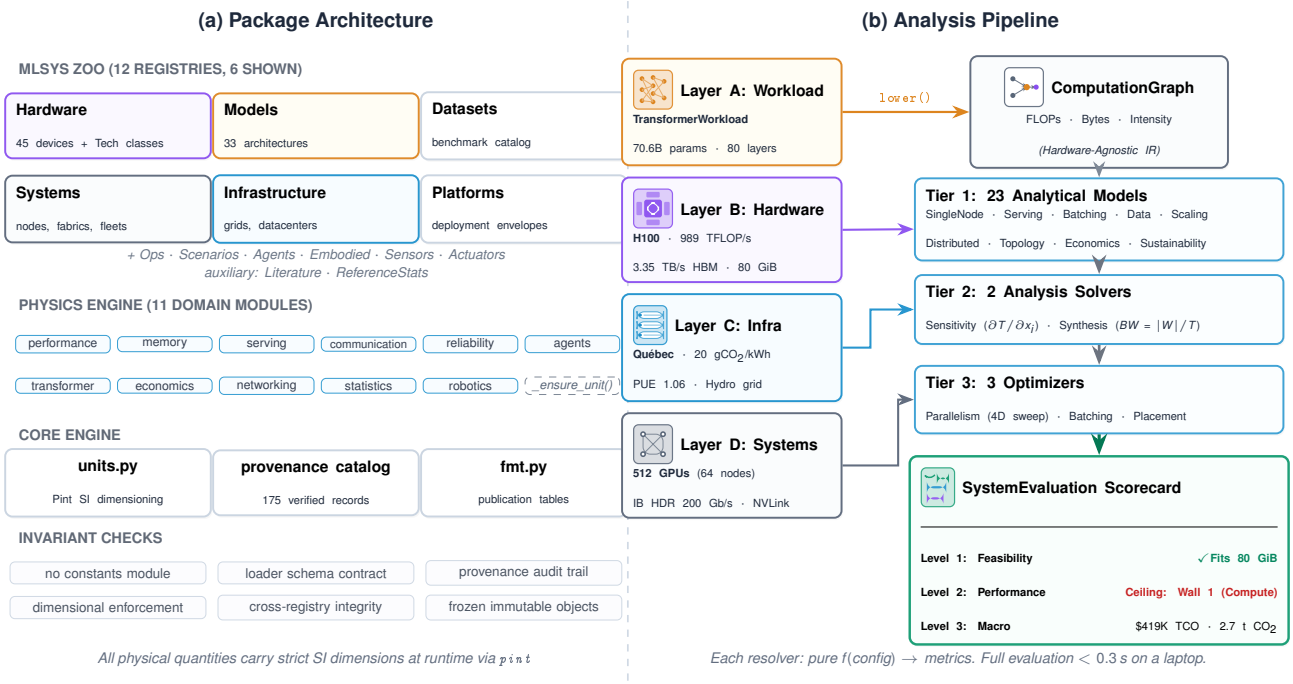


Figure 4: **System Anatomy.** (a) Package architecture: 12 curated registries (six shown) provide typed, provenance-tracked specifications, 27 domain physics modules supply the formulas, and five invariant checks enforce design decisions as the codebase evolves. (b) Analysis pipeline: a workload lowers to a hardware-agnostic intermediate representation; four input layers feed 276 stateless resolvers organized in three tiers; the output is a three-level `SystemEvaluation` scorecard (Feasibility, Performance, Macro).

The `ComputationGraph` IR bridges the demand–supply gap. When a solver calls `workload.lower()`, the workload computes its total operations, weight bytes, and arithmetic intensity, all in dimensioned quantities. For Mixture-of-Experts models, `SparseTransformerWorkload.lower()` uses *active* parameters for FLOPs but *total* parameters for memory footprint, correctly modeling the fundamental decoupling between compute cost and capacity requirements in sparse architectures (Shazeer et al., 2017).

The evaluation produces a `SystemEvaluation` scorecard that groups the metrics of the composed resolvers into Feasibility, Performance, and Macro levels. Students can inspect any individual level or view the aggregate to understand how constraints interact across the stack.

3.4 Extensibility

The layered architecture is designed for extension at every level. New workload types (e.g., a `RetrievalAugmentedWorkload` for RAG pipelines) require implementing the `lower()` method to produce a `ComputationGraph`. Resolvers that consume only the graph, such as the `SingleNodeModel`, then apply unchanged, while architecture-specific resolvers such as the `ServingModel` also require transformer fields. New hardware entries are added to the Hardware registry as declarative `HardwareNode` specifications (Listing 2), with no resolver changes needed. New resolvers can be introduced for emerging constraints by implementing the appropriate tier interface: a Tier 1 Model (e.g., a `PrivacyModel` for federated learning overhead), a Tier 2 Solver, or a Tier 3 Optimizer.

Typed inputs and dimensioned outputs catch schema violations and incompatible units when extensions use the declared interfaces. They cannot detect an incorrect equation, a wrong but dimensionally valid constant, or incompatible modeling assumptions. An extension therefore needs tests for its limiting cases and an explanation of the regime in which its equations apply.

3.5 Public API and Presentation Layer

The package separates analysis from presentation. `from mlsysim import *` exposes the registries, the physics formulas, the unit vocabulary, and the formatting helpers; solver classes are imported through the stable `mlsysim.solvers` path, which

Table 4: **Physics Domain Modules.** Each module owns the analytical formulas for one domain. The three extension modules sit outside the wall taxonomy (Section 9).

Module	Domain	Key Formulas
`performance`	Node	Roofline bottleneck, Amdahl's speedup, pipeline bubble
`memory`	Node	Model memory, activation memory, KV-cache, checkpoint size
`serving`	Node	Erlang-C M/M/c queue latency
`communication`	Fleet	Ring/tree/hierarchical AllReduce, All-to-All
`reliability`	Fleet	Young–Daly, cluster MTBF, failure probability
`transformer`	Algorithm	Training/decode FLOP counts
`economics`	Operations	Fleet TCO, egress cost
`networking`	Data	Speed-of-light latency
`statistics`	Analysis	PSI drift detection, sample sizing
`agents`	Extension	Trajectory reliability, test-time compute cost, multi-agent coordination
`robotics`	Extension	Sensor-to-actuator latency, safe stopping distance, action-chunk cadence
`drivetrain`	Extension	Braking limits (traction, tip-over, drive torque), back-EMF headroom

mechanically re-exports the canonical implementations in `mlsysim.engine.solvers` so the two surfaces cannot drift. The `engine` package is the infrastructure-modeling core: the Roofline core (`engine.engine`), the 276 resolvers, the three-level evaluation, pipeline composition, design-space exploration, executable scenarios, the machine-readable wall taxonomy, calibration defaults, and published benchmark baselines. Presentation lives outside the engine: `mlsysim.fmt` provides a typed `fmt_*` formatter family (94 helpers for quantities, counts, currencies, percentages, and scientific notation) whose outputs are Markdown-safe strings and whose precision-safety guards refuse to render non-finite values or to silently round a non-zero value to the literal "0"; `mlsysim.viz` supplies matplotlib plotters with a consistent style; and a command-line interface (`python -m mlsysim`) exposes the zoo browser, scenario evaluation, serving analysis, design-space optimization, a local-hardware Iron Law audit, and schema export.

Table 5: **Notation.** Symbols used throughout; physical quantities use compatible runtime units; ratios and counts can be dimensionless.

Symbol	Unit	Description
Model & Workload		
P, P_ℓ	params	Total / per-layer parameter count
$\|W\|$, $\|W_\ell\|$	bytes	Bytes read per step / per layer
b_{prec}, b	B/param	Precision (e.g., 2 for FP16) / element bytes
L, H, a	–	Layers, hidden dimension (d_{model}), attention heads
S, S_{in}	tokens	Sequence length / prompt length
B	samples	Batch size
D	tokens	Dataset size ($C = 6PD$)
K	–	Reasoning steps
C	FLOPs	Training compute ($6PD$)
I	FLOP/B	Arithmetic intensity
Hardware & Infrastructure		
$\text{Peak}_{\text{FLOPS}}$	FLOP/s	Peak accelerator throughput
BW_{HBM}	B/s	HBM bandwidth
BW_{inject}	B/s	Injection BW (wafer-scale)
BW_{link}	B/s	Per-link network bandwidth
BW_{storage}	B/s	Persistent storage bandwidth
N, G	–	Nodes in fleet, GPUs per node
α	s	Per-hop network latency
Efficiency & Utilization		
η, η_{silicon}	–	Silicon compute efficiency
MFU, HFU	–	Model / Hardware FLOPs utilization
η_{overlap}	–	Compute–comm overlap
ρ, ρ_{data}	–	Utilization ratio (queue / data pipeline)
β, β_{opt}	–	Bisection BW frac, optimizer multiplier
Parallelism		
TP, PP, DP, EP	–	Tensor, pipeline, data, expert parallel
v, M_{micro}	–	Virtual stages per rank, microbatches
Sustainability		
PUE, WUE	–, L/kWh	Power / Water Usage Effectiveness
CI	gCO_2/kWh	Regional carbon intensity
Key Derived Quantities		
I^*	FLOP/B	Ridge point ($\text{Peak}_{\text{FLOPS}}/BW_{\text{HBM}}$)
B^*, P^*, D^*	varies	Optimal batch, model, dataset size
τ_{opt}	s	Optimal checkpoint interval
δ	s	Checkpoint write duration

4 Taxonomy of ML Systems Walls

The ML systems literature is rich with specialized models for specific bottlenecks, from the original Roofline model (Williams et al., 2009) and Chinchilla scaling laws (Hoffmann et al., 2022) to PagedAttention batching limits (Kwon et al., 2023) and datacenter sustainability accounting (Patterson et al., 2021). However, these constraints are typically studied in isolation. We synthesize this disjointed literature into a unified taxonomy of 20 distinct "Walls" plus 2 cross-cutting diagnostic tools (Sensitivity and Synthesis). We borrow the term from the computer architecture tradition (the "memory wall" (Wulf and McKee, 1995), the "power wall" (Hen-

nessy et al., 2024)), where a *wall* denotes a hard physical or logical constraint that bounds system performance. Within this unified framework, each wall maps to a resolver (model or solver) that accepts typed inputs and produces dimensionally correct bounds. We are not aware of a prior tool that integrates these constraints in one executable engine. The complete taxonomy is summarized in Table 1.

The taxonomy proceeds outward from the silicon. Section 4.1 covers the seven Node walls that bound a single accelerator (compute, memory, software efficiency, serving, batching, streaming, tail latency). Section 4.2 adds the three Data walls that govern how samples reach the accelerator (ingestion, transformation, locality). Section 4.3 covers the three Algorithm walls that originate in the mathematics of learning (complexity, reasoning, fidelity). Section 4.4 adds the three Fleet walls that arise once a job spans multiple nodes (communication, fragility, multi-tenant scheduling). Section 4.5 covers the four Operations walls that determine whether a system should run at all (capital, sustainability, checkpoint, safety). Finally, Section 4.6 introduces the two cross-cutting diagnostic tools (Sensitivity and Synthesis) that operate across the entire taxonomy.

4.1 Node (Single-Accelerator Resources)

The Node walls define what an individual accelerator (such as a single GPU, TPU, or wafer-scale engine) can achieve in isolation, before distributed coordination is considered. Multi-accelerator or multi-node analysis, where a server hosts several accelerators on intra-node links such as NVLink, falls under the Fleet domain (Section 4.4). Node walls are the innermost constraints and the first a practitioner should evaluate.

Wall 1: The Compute Wall. Every accelerator has a hard throughput ceiling determined by the number of arithmetic units and the clock frequency. An H100, for example, provides a peak of 989 TFLOP/s at FP16 with Tensor Cores, establishing an upper bound that no software optimization can exceed. The `SingleNodeModel` evaluates the classical Roofline model bounds (Williams et al., 2009). If execution is entirely compute-bound, the minimum time to process the workload is dictated by the *Iron Law of ML Systems*:

$$T_{\text{compute}} = \frac{\text{OPs}}{\text{Peak}_{\text{FLOPS}} \times \eta} \tag{1}$$

where $\eta \in (0, 1]$ is the declared single-accelerator silicon compute efficiency (η_{silicon}, reflecting physical systolic tile utilization and heterogeneous ALU limits) and OPs is the total operation count. At the cluster level, *Model FLOPS Utilization* (MFU) measures only model-useful FLOPs delivered by the entire fleet, incorporating pipeline schedule bubbles, collective communication latency, and activation recomputation. For a single isolated GEMM without recomputation, $\eta \approx$ MFU; across a distributed cluster, MFU $\leq \eta$, with equality only in the idealized limit of zero communication, bubble, or recomputation overhead. When compute binds, execution time scales directly with operation count and hardware throughput: throughput improves only through higher hardware peak rates, algorithmic operation reduction, or kernel optimizations that raise sustained silicon utilization η toward peak.

Wall 2: The Memory Wall. High-bandwidth memory (HBM) imposes two ceilings: capacity (the model must fit) and bandwidth (weights must stream to compute units fast enough). An H100 reads HBM at 3.35 TB/s, yet its 989 TFLOP/s demand data at a rate that exceeds this bandwidth for any workload below ∼295 flop/byte, making most LLM inference memory-bound, not compute-bound. During training, techniques such as Low-Rank Adaptation (LoRA) and activation recomputation alter the capacity constraint by trading compute or parameter trainability for reduced memory footprints. The same `SingleNodeModel` computes (Williams et al., 2009):

$$T_{\text{memory}} = \frac{|W|}{BW_{\text{HBM}}} \tag{2}$$

where $|W|$ is the total bytes read per inference step. Under ideal overlap, the Roofline estimate takes the maximum of these terms. The implementation adds dispatch and layer overhead separately:

$$T = \max(T_{\text{compute}},\ T_{\text{memory}}) \tag{3}$$

The crossover between compute-bound and memory-bound regimes occurs at the *ridge point*, the arithmetic intensity at which the two ceilings intersect:

$$I^* = \frac{\text{Peak}_{\text{FLOPS}}}{BW_{\text{HBM}}} \quad \text{(flop/byte)} \tag{4}$$

This is the peak ridge point. With compute efficiency η, the modeled crossover is ηI^*: intensities below it are memory bound, and intensities above it are compute bound under the stated traffic and overlap assumptions. For memory-bound workloads, arithmetic units spend clock cycles stalled on weight or activation transfers across the memory bus, rendering peak compute throughput secondary to effective memory bandwidth.

For training, capacity pressure is larger than inference weight residency alone. The `TrainingMemoryModel` exposes the first-order per-accelerator accounting directly:

$$M_{\text{train}} = M_{\text{weights}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}} + M_{\text{comm}}. \tag{5}$$

Mixed-precision Adam defaults to FP16/BF16 weights and gradients plus FP32 master weights and optimizer moments, while tensor, pipeline, expert, and ZeRO sharding reduce different terms of the sum (Shoeybi et al., 2019; Rajbhandari et al., 2020). Activation checkpointing reduces M_{act} by recomputing intermediate tensors during the backward pass; the activation term implements the per-layer analytical bounds of Korthikanti et al. (2023) ($34\,sbh$ bytes per layer with selective recomputation; $34\,sbh + 5as^2b$ without, where s, b, h, a are sequence length, microbatch size, hidden dimension, and attention heads). The model intentionally reports each term separately so users can identify whether a configuration fails because of optimizer state, activations, or communication buffers rather than encountering an opaque out-of-memory error.

Wall 3: The Software Wall. The gap between peak and achieved FLOP/s is typically larger than the gap between hardware generations. Megatron-LM's single-V100 baseline sustained only 30% of theoretical peak FLOPs (Shoeybi et al., 2019), and published MFU for large training runs spans 21.3% to 46.2% (Chowdhery et al., 2022); much of the gap comes from memory traffic, idle kernels, and unfused operations. The `EfficiencyModel` models this as a multiplicative efficiency factor:

$$\eta = \frac{\text{FLOPS}_{\text{achieved}}}{\text{Peak}_{\text{FLOPS}}} \tag{6}$$

where η scales the compute ceiling. Tile utilization, non-matrix operations, kernel fusion, and launch overhead can all affect achieved throughput. The coefficient summarizes such effects; it does not simulate them or attribute a measured loss to a particular cause. The supplied efficiency must therefore be documented for each comparison. Using a cluster-wide utilization value as this input can double-count communication and pipeline losses that the distributed solver already charges separately, as Benchmark 4 illustrates.

Wall 4: The Serving Wall. Prefill processes a prompt, while decode generates successive tokens (Pope et al., 2023). For a 70B model divided ideally across two H100s, a 512-token prompt gives a linear-work prefill estimate of about 73 ms and a weight-only decode floor of about 21 ms per token. These are arithmetic illustrations that omit communication and additional traffic. The single-device `ServingModel` uses the following richer accounting, shown for one request and no cached prefix:

$$T_{\text{prefill}} = \frac{2P \cdot S_{\text{in}} + \text{FLOPs}_{\text{attn}}}{\text{Peak}_{\text{FLOPS}} \times \eta} + T_{\text{dispatch}}, \tag{7}$$

$$T_c = \frac{F_{\text{step}}}{\text{Peak}_{\text{FLOPS}}\eta}, \quad T_m = \frac{|W_{\text{model}}| + |W_{\text{KV}}|}{BW_{\text{HBM}}}, \tag{8}$$

$$T_{\text{decode}} = \max(T_c, T_m) + T_{\text{layer}}. \tag{9}$$

where S_{in} is the input sequence length, P is parameter count, $\text{FLOPs}_{\text{attn}} = 4LHS_{\text{in}}^2$ accounts for pairwise self-attention across L layers and hidden dimension H, and T_{dispatch} is kernel launch overhead.

This formulation reflects the differing computational scaling of the two phases: in prefill, linear projection FLOPs ($2P \cdot S_{\text{in}}$) represent token-to-weight GEMMs (QKV and MLP layers), scaling strictly linearly with prompt length. Pairwise self-attention ($\text{FLOPs}_{\text{attn}}$), however, scales quadratically ($\mathcal{O}(S_{\text{in}}^2)$). For short prompts ($S_{\text{in}} \leq 1$K), attention accounts for $\lesssim 2\%$ of total prefill compute for large models like 70B (and $\sim$3.4% for 8B), so treating prefill as purely linear ($2PS_{\text{in}}$) is an effective rule of thumb. But as context lengths scale, quadratic attention catches up rapidly: under the `ServingModel` accounting for Llama-3 70B, attention arithmetic rises to 13% of total prefill compute at 8,192 tokens and surges to 38% at 32,768—nearly four out of every ten operations. The solver models this quadratic term explicitly so long-context prefill latency is not severely underestimated.

In decode, the memory term reads the resident weights once per step and the filled KV cache for all requests. The compute term includes linear work and both attention matrix products. With batch size B, context length S, layer count L, and hidden dimension H, dense-model work is approximated by $F_{\text{step}} = 2PB + 4LHSB$. The larger of compute and memory time bounds the step, and T_{layer} adds the configured tax across all layers. Each step emits one token per request, so per-stream ITL is T_{decode} while aggregate throughput is B/T_{decode}. Batching amortizes weight reads, but increasing B can make compute time dominate. The same helper supplies this accounting to the continuous-batching model.

The model also exposes an optional chunked-prefill estimate inspired by Sarathi-Serve (Agrawal et al., 2025). When `prefill_chunk_tokens` is set, only the new, uncached prefill work is partitioned into chunks:

$$C_{\text{prefill}} = \max\left(1, \left\lceil \frac{S_{\text{in}} - S_{\text{cache}}}{S_{\text{chunk}}} \right\rceil\right), \tag{10}$$

$$T_{\text{stall}} = \max_i \left(T_{\text{chunk},i}\right) \tag{11}$$

where $T_{\text{chunk},i}$ is the compute time for chunk i plus one dispatch tax. The resulting `ServingResult` reports `prefill_chunks` (C_{prefill}), `prefill_chunk_time` (T_{stall}), and `decode_stall_bound`, which is set to T_{stall} as a first-order bound on how long the slowest prefill chunk can interfere with ongoing decode iterations. This is a coarse analytical bound, not a full scheduler: total TTFT still includes the same prefill compute work plus one dispatch tax per chunk, and the

solver does not claim Sarathi-Serve's stall-free packing behavior.

For deployment sizing, the serving-capacity resolver combines serving latency, continuous-batching capacity, and tail-queueing estimates. Given average generated length G and per-replica token throughput R_{tok}, it estimates request capacity as

$$Q_{\text{replica}} = \frac{R_{\text{tok}}}{G}, \tag{12}$$

$$P99_{\text{request}} \approx T_{\text{base}} + P99_{\text{queue}} \leq T_{\text{target}}. \tag{13}$$

This converts a common serving question—"how many replicas do I need for a QPS and P99 target?"—into a transparent composition of the latency, batching, and queueing walls. The compute efficiency η remains an exposed parameter with the same interpretation as in the Roofline model; the default is a starting point, not a claim of universal serving efficiency.

The solver also represents prefix caching (Zheng et al., 2024), speculative decoding (Leviathan et al., 2023), and disaggregated serving (Patel et al., 2024). Prefix caching removes the cached tokens from prefill work. Speculation charges sequential draft generation and target verification of $K+1$ query positions for K proposals, including the bonus-token distribution; it then divides by the expected emitted-token count. Both models and their KV caches must fit. Disaggregation adds KV transfer time and checks the prefill and decode devices separately. Prefill remains a compute-time approximation; decode explicitly compares compute and memory time. Workspace allocation, tensor-parallel collectives, and runtime scheduling are outside this serving model.

Wall 5: The Batching Wall. When decode is limited by weight traffic, sharing a step across requests increases throughput. The KV-cache allocator determines how many requests can share that step before storage or compute becomes limiting. A static allocator reserves a contiguous slot of $S_{\max}$ tokens for every request, whatever length the request actually reaches. A paged allocator, as in PagedAttention (Kwon et al., 2023), divides the KV budget into blocks of p tokens and gives a request of S tokens $\lceil S/p \rceil$ blocks on demand. With M_{KV} the HBM left after the weights and k the KV bytes per token, the `ContinuousBatchingModel` computes the concurrency each allocator admits:

$$N_{\text{static}} = \left\lfloor \frac{M_{\text{KV}}}{k\, S_{\max}} \right\rfloor, \qquad N_{\text{paged}} = \left\lfloor \frac{\lfloor M_{\text{KV}}/(k\,p) \rfloor}{\mathbb{E}\left[\lceil S/p \rceil\right]} \right\rfloor \tag{14}$$

where request lengths are exponential with mean $\bar{S}$ and cut off at $S_{\max}$, and the scheduler's maximum batch size caps both counts. The unused share of each allocation is $1 - \bar{S}/S_{\max}$ under static reservation and about one partially filled block per request under paging. The memory term is $(|W| + N\bar{S}k)/BW_{\text{HBM}}$. The shared decode helper compares it with compute time and adds layer overhead, as in Equation (7); throughput is N/t_{step}. For Llama-3 8B on an H100 with $S_{\max} = 8{,}192$ and $\bar{S} = 2{,}048$, static reservation admits 65 requests and leaves 75% of each reservation unused, while 16-token pages admit 259 requests with 0.4% unused, a 1.59× gain in decode throughput; 2,048-token pages admit only 165. Kwon et al. profile contiguous pre-allocation and find that only 20.4% to 38.2% of KV-cache memory holds token states, with the remainder lost to reserved slots, internal fragmentation, and external fragmentation, whereas vLLM reaches 96.3% (Kwon et al., 2023). The solver assumes request lengths follow an exponential distribution, and every admitted request is sized at its full length (peak occupancy), making paged capacity conservative. The static baseline represents max-length reservation with no external fragmentation; exact-size and power-of-two contiguous allocators are not modeled. When the batch-size cap binds both allocators, allocation policy does not alter throughput.

Wall 6: The Streaming Wall. Wafer-scale architectures (Lie, 2022) (e.g., Cerebras CS-3) invert the conventional memory hierarchy. Activations reside on-wafer in SRAM while model weights stream from external MemoryX nodes, shifting the bottleneck from HBM bandwidth to injection interconnect bandwidth. The `WeightStreamingModel` models this as:

$$T_{\text{layer}} = \max\left(\frac{|W_\ell|}{BW_{\text{inject}}}, \frac{2P_\ell \times B}{\text{Peak} \times \eta} \right) \tag{15}$$

where $|W_\ell|$ is the layer weight size in bytes, $P_\ell = |W_\ell|/b_{\text{prec}}$ is the parameter count (with b_{prec} bytes per element), and the factor of 2 accounts for the multiply-accumulate FLOPs per parameter. The two terms inside the max represent the injection time (weight delivery) and the compute time (matrix arithmetic) for one layer. When B is small, weight injection dominates and the compute engine sits idle; when B is large, compute dominates and the injection link sits idle. Setting the two terms equal and substituting $|W_\ell| = P_\ell \times b_{\text{prec}}$ yields the optimal batch size $B^* = (b_{\text{prec}} \times \text{Peak} \times \eta)/(2 \times BW_{\text{inject}})$. This result depends only on numerical precision, peak compute, and injection bandwidth, independent of layer size: it defines the operating point where injection and compute balance, maximizing utilization of both resources. The derivation assumes layer weights dominate the injection payload, reserves 10% overhead for working memory, and models within-layer pipelining as ideal.

Wall 7: The Tail Latency Wall. At scale, P99 latency governs user experience, not the median. Fan-out compounds the tail: if a request must collect responses from

100 servers whose 99th-percentile latency is one second, 63% of such requests take more than one second (Dean and Barroso, 2013). The `TailLatencyModel` models inference replicas as an M/M/c queue using the Erlang-C formula (Harchol-Balter, 2013):

$$\mathbb{P}[\text{wait}] = \frac{(c\rho)^c/c! \cdot (1-\rho)^{-1}}{\sum_{k=0}^{c-1}(c\rho)^k/k! + (c\rho)^c/c! \cdot (1-\rho)^{-1}} \tag{16}$$

where c is the number of replicas, $\rho = \lambda/(c\mu)$ is per-server utilization, and λ, μ are arrival and service rates. The implementation evaluates the Erlang-C sum in log space (a log-sum-exp over log-gamma terms), so deep replica pools do not overflow the factorials in Equation (16). P99 latency grows non-linearly as $\rho \to 1$: for a 10-replica deployment, P99 at 80% utilization is $\sim 3\times$ the service time; at 95%, it rises to $\sim 10\times$, making that gap the difference between meeting and violating a latency SLA. The M/M/c queue provides a first-order estimate; bursty arrivals and heavy-tailed service times usually raise tail latency above it, and the reported P99 adds the wait quantile to the *mean* service time. Stability depends only on ρ, so a deployment that is unstable under M/M/c is unstable under any traffic with the same mean rates. To bridge toward more realistic service-time distributions, the solver accepts a `service_time_cv` parameter (coefficient of variation); when cv $\neq 1$, a Kingman correction factor $(c_v^2 + 1)/2$ scales the wait time, capturing the heavier tails of non-exponential service times.

4.2 Data (Movement & Pipelines)

The Data walls govern how data moves *to* the accelerator. A node that is locally unconstrained can still starve if the data pipeline cannot keep pace.

Wall 8: The Ingestion Wall. Storage I/O must supply training samples at the rate the accelerator consumes them. This wall binds most often in vision tasks, where raw images are large, and data stalls of this kind are well documented in DNN training (Mohan et al., 2021). The `DataModel` takes the demanded read rate as an input and computes the demand–supply ratio:

$$\rho_{\text{data}} = \frac{BW_{\text{demand}}}{BW_{\text{supply}}} \tag{17}$$

When $\rho_{\text{data}} > 1$, the accelerator stalls waiting for data. BW_{demand} is the product of batch size, sample size, and step rate; BW_{supply} is the effective throughput of the storage subsystem after accounting for read amplification and caching. For LLM training on tokenized text, BW_{demand} is small compared with image, audio, and video workloads, which are where this wall usually binds. The model assumes storage bandwidth is the binding bottleneck rather than network I/O during single-node training.

Wall 9: The Transformation Wall. JPEG decoding, tokenization, and augmentation execute on CPU cores, not on the accelerator. When the CPU preprocessing pipeline cannot keep pace, the accelerator stalls even if storage bandwidth is abundant. In the scenario of Case S2 (Section 7), each CPU worker decodes and augments $\sim$850 images/s, so 32 workers cap the pipeline at $\sim$27,200 images/s, below what an 8-GPU DGX A100's compute can consume. The `TransformationModel` quantifies this bottleneck (Murray et al., 2021):

$$T_{\text{transform}} = \frac{B \cdot S_{\text{sample}}}{C_{\text{cpu}}} \tag{18}$$

where B is the batch size (in samples), S_{sample} is the size of one sample in bytes, and C_{cpu} is the aggregate CPU preprocessing throughput in bytes/s. For N_{workers} workers that each finish r_{worker} samples/s after all transformations (decode, augment, normalize), $C_{\text{cpu}} = N_{\text{workers}} \cdot r_{\text{worker}} \cdot S_{\text{sample}}$, so $T_{\text{transform}} = B/(N_{\text{workers}} \cdot r_{\text{worker}})$. Preprocessing is modeled as CPU-bound, scaling linearly with worker count up to core saturation.

Wall 10: The Locality Wall. Network topology determines the effective bandwidth available between any two nodes in the cluster. The `TopologyModel` models this through the *bisection bandwidth fraction* β (Leiserson, 1985), which varies by topology: Fat-Tree provides full bisection ($\beta = 1.0$), Dragonfly (Kim et al., 2008) is modeled at an illustrative $\beta \approx 0.85$, and the ring and 3D torus fractions shrink with node count N ($\beta = 2/N$ and $\beta = 2N^{-1/3}$). The effective inter-node bandwidth is:

$$BW_{\text{eff}} = \frac{BW_{\text{link}} \times \beta}{\text{oversubscription}} \tag{19}$$

This wall becomes binding when collective communication patterns demand bandwidth that the topology cannot supply at scale. For example, a 27-node 3D torus with 400 Gb/s links has $\beta \approx 0.67$ and delivers $\sim$267 Gb/s per node, and at 64 nodes β falls to 0.50 (200 Gb/s); a Fat-Tree with the same links provides the full 400 Gb/s ($\beta = 1.0$), a 1.5$\times$ advantage at 27 nodes that grows with cluster size and compounds across multi-hop AllReduce patterns. We place the Locality Wall in the Data domain rather than Fleet because it models the *physical topology constraint* on data movement (bisection bandwidth, oversubscription), whereas the Communication Wall (Wall 14, Fleet) models the *algorithmic cost* of specific collectives (AllReduce, All-to-All) that run atop that topology. The two walls interact but address distinct levels of abstraction. The factor β is modeled as a static function of topology and node count; real networks may exhibit dynamic congestion beyond this static bound.

4.3 Algorithm (Scaling & Compression)

The Algorithm walls arise not from hardware but from the mathematics of learning itself. They determine how much computation a workload *requires*, independent of the silicon that executes it.

Wall 11: The Complexity Wall. Chinchilla scaling laws (Hoffmann et al., 2022) establish that training compute scales jointly with model size P and dataset size D: doubling the parameters requires approximately doubling the tokens to remain compute-optimal. The `ScalingModel` implements:

$$C = 6PD \quad \text{(total training FLOPs)} \tag{20}$$

$$D^* \approx 20P \quad \text{(compute-optimal tokens)} \tag{21}$$

$$P^* = \sqrt{\frac{C}{120}} \quad \text{(optimal model size for budget } C\text{)} \tag{22}$$

These relations allow system designers to reason backward from a compute budget to the largest model that can be trained optimally, or forward from a model size to the minimum viable training cluster. For example, a budget of 10^{24} FLOPs yields $P^* \approx 91$B parameters with $D^* \approx 1.8$T tokens ($D^* \approx 20P^*$). Chinchilla (70B parameters, 1.4T tokens) was trained near this optimal frontier at a budget of $C \approx 5.88 \times 10^{23}$ FLOPs (Hoffmann et al., 2022). The optimal token-to-parameter ratio ($D^*/P^* \approx 20$) means that doubling compute requires scaling both parameters and tokens by $\sqrt{2} \approx 1.41\times$, rather than allocating compute solely to model size. Scaling law coefficients are fitted to published training runs; the `ScalingModel` does not flag extrapolation beyond that range, so budgets far outside published runs deserve caution.

Wall 12: The Reasoning Wall. Inference-time compute scaling introduces a cost that grows linearly with the number of reasoning steps K. The `InferenceScalingModel` models this as (Snell et al., 2025):

$$T_{\text{reason}} = K \times T_{\text{step}}(P, S_{\text{context}}) \tag{23}$$

where T_{step} is the per-step latency, itself a function of model size P and context length S_{context}. In practice, K varies by strategy: best-of-N sampling uses $K = N$, and Snell et al. (2025) evaluate best-of-N, beam search against a process reward model, and sequential revisions at generation budgets of up to 256. A K=8 query spends $\sim 8\times$ the decode compute of a single-pass answer, substantially altering serving economics (see Case S1, Section 7). The model assumes the prompt is prefilled once and the K steps are decoded at a fixed context length, ignoring intermediate context growth across steps.

Wall 13: The Fidelity Wall. Compression trades model fidelity for efficiency. Quantization reduces precision while pruning removes weights entirely. The accuracy–efficiency frontier is task- and architecture-dependent. The `CompressionModel` quantifies the two primary mechanisms (Han et al., 2016; Gholami et al., 2022):

$$r_{\text{quant}} = \frac{b_{\text{base}}}{b_{\text{target}}} \quad \text{(e.g., } 16/4 = 4\times \text{ vs. FP16)} \tag{24}$$

$$r_{\text{prune}} = \frac{1}{1-s} \quad \text{(memory reduction at sparsity } s\text{)} \tag{25}$$

Here b_{base} is a configurable baseline precision and b_{target} is the quantized precision. The `CompressionModel` defaults to an FP32 baseline, so INT4 reports $8\times$ unless the caller passes `baseline_precision="fp16"`, which gives the $4\times$ reduction relative to the FP16 or BF16 weights that models are usually served from. Quantization reduces memory traffic without changing FLOP counts, shifting arithmetic intensity rightward on the Roofline by a factor of r_{quant} and potentially crossing the ridge point from memory-bound to compute-bound. Post-training quantization methods such as GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) demonstrate that 4-bit quantization preserves most accuracy for large models, making this $4\times$ weight reduction a standard production choice. For pruning, r_{prune} measures memory reduction, but memory savings alone do not determine execution speedup: unstructured pruning reduces storage without accelerating standard GPU GEMM kernels, whereas N:M structured sparsity (e.g., 2:4 on NVIDIA Ampere and Hopper architectures) can double throughput on supported hardware (NVIDIA Corporation, 2023). The `CompressionModel` reports both `memory_savings_pct` and `inference_speedup` to capture this divergence. Accuracy degradation Δ_{acc} is estimated using configurable heuristic curves (Section 9), as the empirical frontier depends on workload and task.

4.4 Fleet (Multi-Node Coordination)

The Fleet walls arise when systems scale beyond a single node, requiring coordination across multiple accelerators connected by network fabric.

Wall 14: The Communication Wall. Distributed training requires gradient synchronization across N nodes, and the cost of that synchronization grows with both message size and node count. The `DistributedModel` models the dominant collective operations. For Ring AllReduce (Patarasuk and Yuan, 2009) and its ZeRO/FSDP partitioned equivalents (Reduce-Scatter and All-Gather):

$$T_{\text{ring}} = \frac{2(N-1)}{N} \cdot \frac{M}{BW_{\text{link}}} + 2(N-1) \cdot \alpha \tag{26}$$

where M is the message size, BW_{link} is the per-link network bandwidth, and α is the per-hop latency. Modern clusters use hierarchical AllReduce to exploit the

bandwidth asymmetry between intra-node interconnect (e.g., NVLink at 450 GB/s per direction, quoted as 900 GB/s bidirectional) and inter-node fabric (e.g., InfiniBand NDR at 50 GB/s per port, a 9× gap). The `DistributedModel` implements a two-level model:

$$T_{\text{hier}} = T_{\text{intra}}(BW_{\text{NVLink}}, G) + T_{\text{inter}}(BW_{\text{IB}}, N) \quad (27)$$

where G is GPUs per node and N is the node count. Each level applies the ring formula (Equation (26)) at its respective bandwidth, making the inter-node phase the dominant cost at scale. After the intra-node reduce-scatter, every GPU holds an M/G shard of the partial reduction; G inter-node rings then operate in parallel on M/G payloads rather than full M buffers, amortizing inter-node traffic. In practice, modern frameworks also use compute–communication overlap, hiding network latency behind backward-pass computation. The `DistributedModel` models this with an explicit overlap efficiency parameter $\eta_{\text{overlap}} \in [0, 1]$ (default 0.85, representing achievable overlap between non-blocking AllReduce and backward compute), yielding an exposed communication cost of $(1 - \eta_{\text{overlap}}) \cdot T_{\text{comm}}$. Gradient accumulation further reduces effective communication cost. When accumulating gradients over K_{acc} microsteps before synchronizing, the DP AllReduce cost is amortized by $1/K_{\text{acc}}$, trading latency for communication efficiency at large global batch sizes. Tensor parallelism partitions model weights across devices and requires two AllReduce operations per transformer layer on *activations* (not weights) in the forward pass and two more in the backward pass (Shoeybi et al., 2019). Each AllReduce exchanges a tensor of size $B \times S \times H \times b$ bytes, where B is batch size, S is sequence length, H is hidden dimension, and b is bytes per element. Weights are partitioned once and remain stationary. Pipeline parallelism (Narayanan et al., 2021) introduces a bubble overhead:

$$B_{\text{pipeline}} = \frac{P_{\text{stages}} - 1}{v \cdot M_{\text{micro}} + P_{\text{stages}} - 1} \quad (28)$$

where v is the number of virtual stages per pipeline rank ($v = 1$ for non-interleaved 1F1B) and M_{micro} is the number of microbatches. For Mixture-of-Experts All-to-All dispatch:

$$T_{\text{a2a}} = \frac{(N-1)}{N} \cdot \frac{M}{BW_{\text{link}}} + (N-1) \cdot \alpha \quad (29)$$

where M is the routed activation payload. The `MoERoutingModel` maintains a first-order MoE abstraction where the total parameter count determines memory residency, the active parameter count determines the compute workload, and any hot-expert imbalance inflates the routed payload. With routing imbalance factor $\gamma \geq 1$:

$$E_{\text{active,eff}} = \min(E_{\text{total}}, \gamma \cdot E_{\text{top-k}}), \quad (30)$$

$$P_{\text{active,eff}} = P_{\text{active}} \cdot \frac{E_{\text{active,eff}}}{E_{\text{top-k}}}. \quad (31)$$

The same factor is available in `DistributedModel` as `moe_routing_imbalance_factor`, where it inflates expert-parallel traffic without introducing a router simulation. This follows the load-balancing motivation of sparsely-gated MoE and Switch-style routing systems (Shazeer et al., 2017; Fedus et al., 2022; Lepikhin et al., 2020).

Wall 15: The Fragility Wall. Component failures are inevitable at scale. If each node has a mean time between failures of $\text{MTBF}_{\text{node}}$, then a cluster of N nodes has (Daly, 2006):

$$\text{MTBF}_{\text{cluster}} = \frac{\text{MTBF}_{\text{node}}}{N} \quad (32)$$

The probability of at least one failure during a training run of duration T is:

$$P(\geq 1 \text{ failure}) = 1 - e^{-T/\text{MTBF}_{\text{cluster}}} \quad (33)$$

The Young–Daly formula (Young, 1974; Daly, 2006) gives the optimal checkpoint interval:

$$\tau_{\text{opt}} = \sqrt{2\delta \cdot \text{MTBF}_{\text{cluster}}} \quad (34)$$

where δ is the time to write one checkpoint. The `ReliabilityModel` uses these relations to estimate the fraction of compute lost to checkpointing and recovery, reporting a `goodput_ratio` that captures the effective fraction of wall-clock time spent on useful forward/backward computation after subtracting checkpoint writes (δ/τ_{opt}) and failure recovery ($R/\text{MTBF}_{\text{cluster}}$). Under the memoryless Poisson failure model, expected rework between the last checkpoint and a failure averages $\tau_{\text{opt}}/2$, which at the Young–Daly optimum equals the checkpoint overhead δ/τ_{opt}. The framework's basic goodput equation tracks checkpoint and restart time, and users can include the rework term for full Young–Daly loss accounting.

Wall 16: The Multi-tenant Wall. Production GPU clusters are rarely dedicated to a single job. Shared clusters introduce queueing delays that grow hyperbolically as utilization approaches 1.0, creating a tension between maximizing hardware utilization and minimizing researcher wait times. The `OrchestrationModel` treats the whole cluster as a single server and models job wait times with the M/D/1 mean wait (Harchol-Balter, 2013):

$$T_{\text{wait}} = \frac{\rho}{2\mu(1-\rho)} \quad (35)$$

where $\rho = \lambda/\mu$ is the cluster utilization, λ is the job arrival rate, and μ is the service rate. As $\rho \to 1$, wait times diverge hyperbolically. A cluster at 90% utilization has 9.0× the mean wait and 16.2× the mean queue length of one at 50%. Job durations are modeled as approximately deterministic, which is reasonable for large training runs with predictable step times.

4.5 Operations (Cost, Carbon & Safety)

The Operations walls capture constraints that are not about *how fast* a system runs but *whether it should run at all*: economic viability, environmental impact, checkpoint overhead, and responsible deployment.

Wall 17: The Capital Wall. Performance analysis is incomplete without economic constraints. A 1,024-GPU H100 cluster costs ∼$25.6M in CapEx alone; amortized over 3 years, the hardware cost of a single 30-day training run is about $701K before adding electricity. The `EconomicsModel` computes total cost of ownership (Barroso et al., 2019):

$$\text{TCO} = \text{CapEx} + \text{OpEx}_{\text{energy}} + \text{OpEx}_{\text{maint}} \tag{36}$$

where $\text{OpEx}_{\text{energy}} = E_{\text{total}} \times P_{\text{kWh}}$ converts total energy consumption to dollar cost at the regional electricity price. At scale, CapEx dominates. For that run, electricity at $0.12/kWh is 7.9% of the $876K total, making hardware utilization (MFU) the primary lever for cost efficiency. Amortization follows a linear schedule over a 3–5 year hardware lifetime.

Wall 18: The Sustainability Wall. Regional grid carbon intensity varies by more than an order of magnitude, making geography a first-order systems design variable. The `SustainabilityModel` converts energy into environmental impact (Patterson et al., 2021):

$$E_{\text{total}} = E_{\text{IT}} \times \text{PUE} \tag{37}$$

$$\text{CO}_2 = E_{\text{total}} \times \text{CI}_{\text{region}} \quad (\text{gCO}_2/\text{kWh}) \tag{38}$$

$$\text{H}_2\text{O} = E_{\text{total}} \times \text{WUE} \quad (\text{L/kWh}) \tag{39}$$

where PUE is the power usage effectiveness of the datacenter, $\text{CI}_{\text{region}}$ is the carbon intensity of the local grid, and WUE is the water usage effectiveness. This model captures *operational* carbon only. For battery-powered consumer devices, *embodied* carbon from manufacturing often dominates, and Gupta et al. (2021) attribute roughly 75% of such devices' life-cycle emissions to manufacturing and about 20% to energy use. The `SustainabilityModel` accepts an optional `embodied_carbon_per_device` parameter for lifecycle analysis. Grid carbon intensity is treated as a static regional constant, omitting temporal renewable intermittency. Power is modeled as an idle floor of 30% of TDP (reflecting modern accelerator low-power states) plus a dynamic component that scales linearly with MFU; historically, earlier CPU servers drew closer to half of their peak power at idle (Barroso and Hölzle, 2007).

Wall 19: The Checkpoint Wall. Long-running training jobs must periodically save model state (weights and optimizer states) to persistent storage, incurring an I/O penalty that directly reduces sustained efficiency. Because checkpoint writes are serial stalls in an otherwise parallel job, their share of wall-clock time caps effective MFU. The `CheckpointModel` models this I/O burst penalty, a first-order version of the checkpoint cost that systems such as Check-N-Run (Eisenman et al., 2022) work to reduce:

$$\text{MFU}_{\text{penalty}} = \frac{T_{\text{write}}}{T_{\text{interval}}} = \frac{|W| \times \beta_{\text{opt}} / BW_{\text{storage}}}{T_{\text{ckpt_interval}}} \tag{40}$$

where $|W|$ is the model weight size in bytes, and β_{opt} is the optimizer state multiplier, the ratio of total checkpoint bytes to model weight bytes. For mixed-precision Adam with FP16 weights, $\beta_{\text{opt}} \approx 7$ (FP32 master weights at 4 bytes + FP32 momentum at 4 bytes + FP32 variance at 4 bytes, plus FP16 model weights at 2 bytes, totaling 14 bytes per parameter vs. 2 bytes for the FP16 model alone). Gradients are ephemeral and not checkpointed. For a 70B-parameter model, the checkpoint size is 71B × 14 B/param ≈ 1 TB, making storage bandwidth the binding constraint during I/O bursts.

Wall 20: The Safety Wall. Privacy and fairness guarantees impose quantifiable computational overhead. The `ResponsibleEngineeringModel` models the cost of differential privacy via DP-SGD (Abadi et al., 2016), where the noise multiplier scales inversely with the privacy budget:

$$\sigma \propto \frac{1}{\varepsilon} \tag{41}$$

Per-sample gradient clipping and noise addition slow DP-SGD training, and the solver models the slowdown as $T_{\text{DP}} = T_{\text{base}}(1 + c/\varepsilon)$ with an illustrative coefficient $c = 2$, which gives 3× at $\varepsilon = 1$ and 21× at $\varepsilon = 0.1$; c is not fitted to published measurements. Fairness constraints impose a complementary data requirement: sufficient representation of minority subgroups demands additional training data proportional to $O(1/p_{\text{min}})$, where p_{min} is the smallest subgroup prevalence. Unlike preceding hardware walls, Wall 20 captures constraints where overhead is heavily dependent on specific algorithmic formulations. The solver treats the privacy budget ε as an input constraint, reporting an illustrative compute multiplier rather than certifying a mathematical guarantee.

4.6 Analysis (Cross-Cutting Diagnostics)

The preceding 20 walls each model a specific physical or logical constraint. The final two entries are *diagnostic tools* rather than walls in the strict sense. They operate *across* the taxonomy rather than within a single domain, providing analysis capabilities that span all walls. The Sensitivity tool identifies which wall is binding, and the Synthesis tool derives minimum hardware from SLA requirements.

Wall 21: The Sensitivity Wall. Optimization is effective only when directed at the binding constraint; improving a non-bottleneck parameter yields no measurable gain. The `SensitivitySolver` identifies the binding constraint by estimating the sensitivity of end-to-end latency to each hardware parameter:

$$\frac{\partial T}{\partial BW_{\text{mem}}}, \quad \frac{\partial T}{\partial \text{Peak}_{\text{FLOPS}}}, \quad \frac{\partial T}{\partial M_{\text{cap}}} \tag{42}$$

The parameter with the largest sensitivity is the binding constraint: the single upgrade that yields the greatest performance improvement. Each reported sensitivity is the normalized latency response $(T_{\text{perturbed}} - T_{\text{base}})/T_{\text{base}}$ to a 10% parameter upgrade. For Llama-3 8B decode on an A100, the solver reports a memory-bandwidth sensitivity of -0.088 (a 10% bandwidth upgrade cuts latency by 8.8%) versus exactly zero for peak FLOPS, because under the hard max of Equation (3) a strictly memory-bound workload does not respond to a compute upgrade at all. This transforms "where should I invest?" from intuition into calculation. Sensitivities use a finite-difference approximation with 10% perturbation, ignoring second-order coupling.

Wall 22: The Synthesis Wall. The `SynthesisSolver` addresses the inverse problem: given a service-level objective (e.g., 50 ms inter-token latency), it inverts the Roofline bounds (Williams et al., 2009) to derive the minimum hardware specifications required to satisfy it:

$$BW_{\text{required}} = \frac{|W|}{T_{\text{target}}} \tag{43}$$

$$\text{FLOPS}_{\text{required}} = \frac{\text{OPs}}{T_{\text{target}} \times \eta} \tag{44}$$

This enables hardware-software co-design. Engineers specify an SLA and the solver derives the minimum hardware that satisfies it. For Llama-3 70B at a 50 ms inter-token latency target, the solver yields $BW_{\text{required}} = 141.2\ \text{GB}/50\ \text{ms} \approx 2{,}824\ \text{GB/s}$, $1.38\times$ the A100's 2.04 TB/s, confirming that this SLA requires at least 2 tensor-parallel A100s. Hardware parameters are treated as independently adjustable design targets; co-design coupling between FLOPS and bandwidth is not modeled.

5 The 3-Tier Resolver Architecture

The walls define *what* constrains a system. This section formalizes *how* resolvers compose to produce end-to-end system evaluations. To clarify the mathematical intent of each component, MLSys·im organizes its analytical tools into a 3-tier taxonomy. Below, we introduce each tier; Section 5.1 then shows how tiers chain into multi-stage analyses through a shared typed contract, and Section 5.2 describes the resulting three-level `SystemEvaluation` scorecard (Feasibility, Performance, Macro).

Tier 1: Analytical Models (the "physics engine"). Models perform forward evaluation $Y = f(X)$ to determine the physical and logical consequences of a specific system configuration. For example, the `ServingModel` estimates the time-to-first-token for a given LLM and GPU pair. Models are purely deterministic and make no decisions; they make up 271 of the 276 resolvers in our taxonomy.

Tier 2: Analysis Solvers (the "math engine"). Solvers perform algebraic inversion, $X = f^{-1}(Y)$, or finite-difference sensitivity analysis to find the parameter required to hit a specific target or the parameter that binds. For example, the `SynthesisSolver` takes a target latency SLA and works backward to derive the minimum memory bandwidth required.

Tier 3: Optimizers (the "engineering engine"). Optimizers perform constrained design-space search, $\max f(X)$ subject to $g(X) \leq c$, to find the best configuration among many valid options. Unlike Models and Solvers, which map directly to individual walls, Optimizers operate across the entire taxonomy. The `ParallelismOptimizer` sweeps power-of-two tensor and pipeline degrees (TP capped at the node size by default, with data parallelism filling the remainder) to maximize Model FLOPs Utilization (MFU) on a given cluster; the `BatchingOptimizer` searches for the maximum batch size that satisfies a P99 queueing latency SLA.

5.1 Stateless Composition and Chaining

The core analytical resolvers accept a configuration and return a result object with units. A chain passes selected outputs to the next model, but the caller must also preserve the meaning of those outputs. For example, a duration must refer to the same workload and device count used by the cost model. Units catch incompatible dimensions; they do not detect an inconsistent scenario.

The following chain connects a scaling-law allocation to distributed execution and then to fleet cost:

$$\text{Scaling} \xrightarrow{\mathcal{R}_1} \text{Distributed} \xrightarrow{\mathcal{R}_2} \text{Economics} \quad (45)$$

Listing 3 demonstrates this exact chain in MLSYS·IM. The `ScalingModel` calculates the optimal model size for a given compute budget and hands that workload to the `DistributedModel` ($\mathcal{R}_1$), which computes the execution time on an 8,192-GPU fleet, factoring in 3D parallelism overhead. The `EconomicsModel` then converts that execution time into a Total Cost of Ownership ($\mathcal{R}_2$); a `SustainabilityModel` call can extend the same chain to carbon.

Each link in the chain preserves dimensional correctness. Units propagate through the computation, and any mismatch raises an immediate error rather than producing a silently wrong result.

5.2 The SystemEvaluation Scorecard

MLSYS·IM provides a `Scenario.evaluate()` entry point that orchestrates solver composition automatically through a three-level evaluation. A scenario is a concrete executable case, such as a doorbell workload on a microcontroller or a frontier training workload on a fleet; sourced statistics used only as reference baselines remain outside the executable path in `ReferenceStats.*`.

Level 1: Feasibility. Does the model fit? Can the data pipeline keep pace? The framework checks memory capacity against model size, ingestion bandwidth against training throughput, and reports any wall where demand exceeds supply.

Level 2: Performance. What are the achievable latency, throughput, and utilization? The Roofline analysis (Equation (3)), communication modeling (Equation (26)), and pipeline bubble (Equation (28)) combine to produce end-to-end training step time.

Level 3: Macro. What does it cost, and what does it emit? The `EconomicsModel` estimates one year of TCO (Equation (36)) and carbon (Equation (37)) for the scenario's fleet.

All three levels are computed and reported, so a Level 1 failure appears alongside the performance and cost figures it invalidates, while `Scenario.validate()` raises on the first failing level. Level 1 compares weight bytes against one accelerator's capacity and checks declared data rates with the `DataModel`. Reading the levels in order reflects the dependency structure, since communication optimization is irrelevant if the model exceeds available memory. The complete implementation details and key assumptions for each solver are documented alongside their respective walls in Section 4.

```python
import mlsysim
from mlsysim.solvers import ScalingModel, DistributedModel, EconomicsModel
from mlsysim.models.types import TransformerWorkload

# 1. Algorithm: Find optimal parameters for a fixed compute budget
budget = mlsysim.Q_("4e24 flop")  # ~100K H100-days at 50% MFU
optimal = ScalingModel().solve(compute_budget=budget)  # P* ~ 183B params

# Instantiate the demand (Layer A: Workload)
model = TransformerWorkload(
    name="Frontier-Model", architecture="transformer",
    parameters=optimal.optimal_parameters,
    layers=80, hidden_dim=8192, heads=64
)

# 2. Fleet: Evaluate on a massive 8K GPU cluster (Layer D: Supply/Topology)
fleet = mlsysim.Systems.Clusters.Frontier_8K
seq_len = 4096
perf = DistributedModel().solve(
    model, fleet,
    batch_size=4096, tp_size=8, pp_size=4, seq_len=seq_len,
    microbatch_count=16,  # one sequence per microbatch
)

# 3. The Capital: Calculate TCO for the resulting training time
n_steps = optimal.optimal_tokens.to("count").magnitude / (4096 * seq_len)
duration = perf.step_latency_total * n_steps
tco = EconomicsModel().solve(fleet, duration_days=duration.to("day").magnitude)

print(f"Scaling Efficiency: {perf.scaling_efficiency:.1%}")  # 73.4%
print(f"Training Duration: {duration.to('day'):.1f}")  # 15.6 day
print(f"Total Job Cost: ${tco.tco_usd:,.0f}")  # $3,637,644
```

Listing 3: **Solver Composition.** Bridging algorithmic scaling, distributed execution, and fleet economics in a single executable chain.

6 Evaluation

The evaluation asks whether the implementation reproduces selected accounting relationships, how its estimates compare with published performance, and how quickly it evaluates configurations. These are different tests. Agreement with a formula checks its implementation; agreement with a measured run also depends on the workload, runtime, topology, and efficiency inputs.

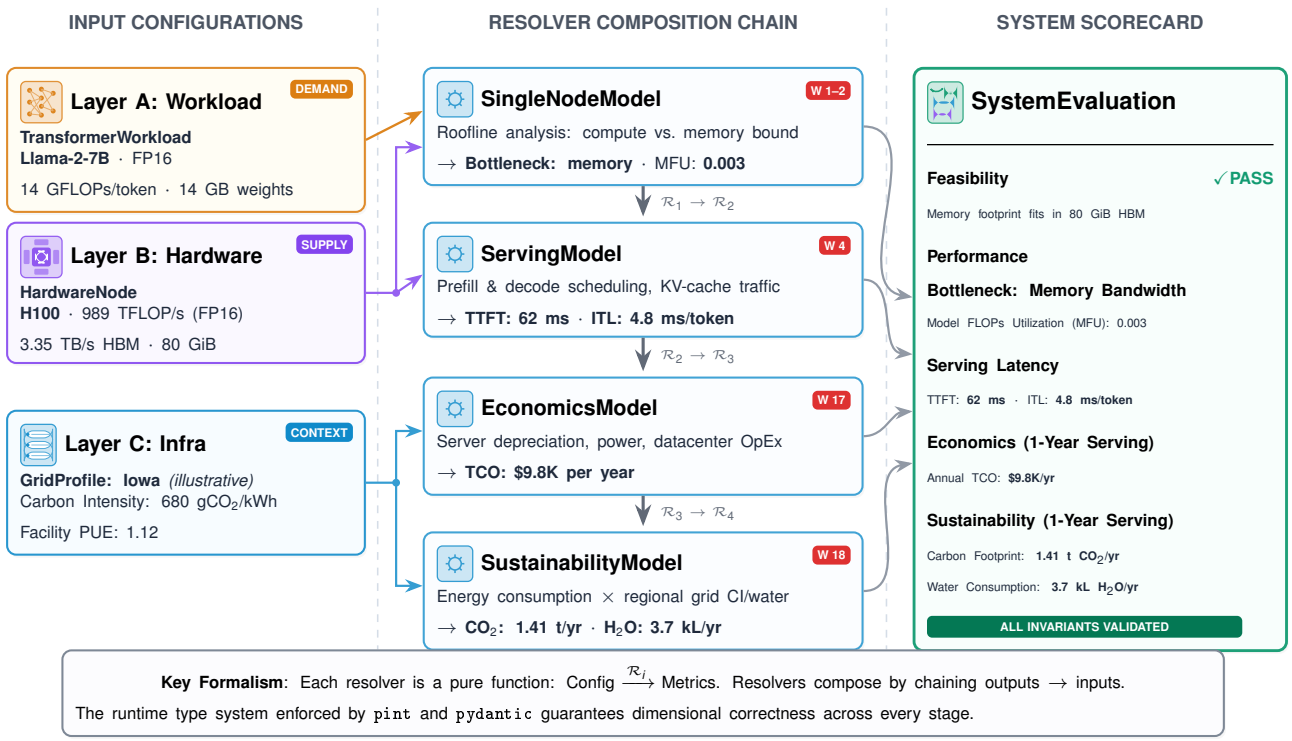


Figure 5: **Resolver Composition.** Three input layers feed four resolvers. Resolvers accept typed inputs and return quantities with units; composition still requires compatible assumptions. The scorecard aggregates three evaluation levels: Feasibility, Performance, and Macro (economics, sustainability, safety).

A discrepancy must be explained or left unresolved, rather than accepted merely because the model is first order.

Section 6.1 separates the evidence for each comparison. Section 6.2 reports local execution time, and Section 6.3 states the principal assumptions. The structural checks in Section 6.4 address software consistency rather than empirical accuracy.

6.1 Published Benchmarks

The seven cases below include performance comparisons, a bandwidth floor, accounting checks, and a search result. The generator records configurations and computes the displayed values from the checked-out package. The validation script retains regression thresholds for selected comparisons and checks the stated identities and search result. Neither a passing regression nor an assumed efficiency establishes accuracy on a held-out workload. In particular, the single-device coefficient η is an input, not an output of the performance model.

Benchmark 1: ResNet-50 on DGX A100 (Single-Node Training). For ResNet-50 on eight A100 GPUs at global batch 2,048, the model evaluates 256 images per device with efficiency $\eta = 0.19$. It predicts 2,376 samples/s per GPU and scales that result ideally across the node to 19,007 samples/s. NVIDIA reports 17,400 images/s with mixed precision and XLA (NVIDIA Corporation, 2020), giving a difference of 9.2%. This checks the configured model against one reported run. The efficiency is supplied rather than independently derived, and ideal node scaling omits synchronization costs; the agreement therefore does not establish a held-out throughput error.

Benchmark 2: Llama-2-70B Decode on H100 (Inference Floor). For Llama-2-70B in FP16 at batch size one, the registry gives 140 GB of weights. With TP=4, ideal aggregate H100 bandwidth gives a weight-read floor of $140\ \text{GB}/(4 \times 3.35\ \text{TB/s}) = 10.45$ ms per step. The floor excludes KV-cache traffic, collective communication, and runtime overhead; TP here matches the comparison configuration rather than the minimum number of devices needed to hold the weights.

LLM-Inference-Bench publishes request timings for this model on four H100s (Chitty-Venkata et al., 2024). At 128 input and 128 output tokens, dividing request latency by output length gives 18.52 ms per output token for vLLM and 18.04 ms for TensorRT-LLM. The

benchmark timers enclose the whole generation call, so these averages include prefill. The CSV provides no time-to-first-token field from which to isolate decode ITL. We therefore report the observed request averages separately and leave the decode-error column unset. The selected source rows and their pinned repository revision are retained in `paper/data/`. This comparison does not identify sustained decode bandwidth utilization.

Benchmark 3: Llama 3 Training at 16K H100s (Distributed Training). Meta's Llama 3 report (Llama Team, AI Meta, 2024) documents 41% BF16 MFU for 405B pre-training on 16,384 H100 GPUs with TP=8, PP=16, and DP=128 at an 8,192-token sequence length, on a 400 Gb/s network fabric, with fully sharded data parallelism for optimizer states and gradients and without activation checkpointing. We configure MLSys·IM's `DistributedModel` with Meta's published layout (TP=8, PP=16, DP=128, ZeRO stage 2), 2,048 sequences of 8,192 tokens per step (yielding 16 microbatches per DP group), InfiniBand NDR fabric (400 Gb/s), declared overlap parameter $\eta_{\text{overlap}} = 0.85$, and declared single-device GEMM efficiency $\eta = 0.50$. To minimize idle pipeline time across 16 pipeline ranks, Meta employs an interleaved 1F1B schedule; with 126 transformer layers across 16 pipeline stages, we assume the maximal interleaving of one layer per virtual stage ($v \approx 8$ virtual stages per rank). Analytically, interleaving divides the pipeline bubble fraction from $\frac{P-1}{M+P-1} = 48.4\%$ (un-interleaved, $v = 1$, which predicts only 24.4% MFU) down to $\frac{P-1}{vM+P-1} = 10.5\%$ ($v = 8$), adding 0.6 s of bubble latency to the 5.0 s compute phase (9.5% of the 6.2 s step). Incorporating this schedule, MLSys·IM predicts an aggregate cluster MFU of 40.6%, matching Meta's reported 41% within 1.0% (an absolute difference of 0.4 percentage points). Sensitivity to the declared η is approximately linear (MFU $\approx 0.74\,\eta$; sweeping η from 0.45 to 0.55 yields MFU from 36.9% to 44.2%). Under MLSys·IM's conservative activation accounting without dynamic tensor deallocation, memory footprint reaches 88.3 GB per GPU (2.8% above the 85.9 GB physical capacity), suggesting that Meta's runtime tensor deallocation as virtual stages retire is needed to fit without checkpointing. We note that MLSys·IM's first-order pipeline model omits point-to-point transfer latency between pipeline stages, which is largely hidden behind compute at this batch size.

Benchmark 4: PaLM HFU-to-MFU Accounting (Pod-Scale Training). Google's PaLM-540B (Chowdhery et al., 2022) was trained without pipeline parallelism on 6,144 TPU v4 chips across two pods. The PaLM report explicitly introduced the distinction between hardware FLOPs utilization (HFU, reported at 57.8%) and model FLOPs utilization (MFU, reported at 46.2%). The difference is governed by activation rematerialization: PaLM's reported ratio 46.2%/57.8% ≈ 0.80 implies that rematerialization adds ≈25% hardware FLOPs over the $6N$ model FLOPs, above the 0.75 ratio of full recomputation. This benchmark evaluates the framework's HFU-to-MFU accounting as a two-step decomposition. Treating reported cluster HFU as the single-device compute baseline ($\eta = 0.58$) and applying full activation recomputation (a $3/4 = 0.75$ FLOP factor, compared to PaLM's ∼0.80 selective rematerialization) alongside cross-pod DCN bandwidth (81 Tbps aggregate burst across pods), MLSys·IM predicts 41.1% MFU ($57.8\% \times 0.75 \times 94.9\% = 41.1\%$), within 11.0% of the reported 46.2% (5.1 percentage points). The 11.0% gap decomposes cleanly into two effects: −2.9 percentage points from modeling full rather than selective recomputation (0.75 vs. ∼0.80), and −2.2 percentage points from communication overhead (5.1% of step time across ICI and DCN) being applied on top of an empirical HFU that already reflected network stalls. This comparison demonstrates why analytical accounting must separate single-device execution from cluster communication rather than conflating cluster HFU with silicon efficiency.

Benchmark 5: Chinchilla Scaling Law (Implementation Consistency). The Chinchilla paper (Hoffmann et al., 2022) establishes that compute-optimal training uses about 20 tokens per parameter. MLSys·IM's `ScalingModel` uses the compute approximation $C = 6PD$ and derives the optimal allocation $P^* = \sqrt{C/120}$. For Chinchilla's own budget, $C = 6 \times 70\text{B} \times 1.4\text{T} \approx 5.88 \times 10^{23}$ FLOPs, the solver returns $P^* = 70.0\text{B}$ and $D^* = 1.40\text{T}$. This is an implementation consistency check rather than an independent empirical comparison, because Chinchilla's 1.4T tokens are exactly 20 times its 70B parameters, so P^* returns 70B by construction.

Benchmark 6: Training Carbon Footprint (Unit Consistency). Patterson et al. (2021) estimate that training GPT-3 on V100 GPUs consumed 1,287 MWh and emitted 552 tonnes CO_2e using the 2020 US average grid intensity (429 gCO_2e/kWh). MLSys·IM's `carbon_from_energy` reproduces 552.1 tonnes from those inputs. Since the published figure is itself this product, the check verifies unit handling, not emissions accuracy.

Benchmark 7: Llama 3 Parallelism Strategy (Optimizer Search). To exercise the Tier 3 design-space search, we configure the `ParallelismOptimizer` with the Llama 3 405B model, its 16,384-GPU H100 cluster (Llama Team, AI Meta, 2024), and Meta's 8,192-token sequences. The optimizer sweeps power-of-two TP and PP degrees, pre-screens each candidate so that the per-GPU

shard of weights, gradients, and Adam optimizer state fits in 90% of HBM, prices each survivor with the `DistributedModel`, drops any whose training state no longer fits once activations are counted, and ranks the 14 remaining candidates by MFU. Because MLSYS·IM's activation accounting admits no split at this sequence length without recomputation (Benchmark 3), the search runs with full activation recomputation. It selects TP=8, PP=16, DP=128, the TP=8, CP=1, PP=16, DP=128 layout Meta reports for this stage; Meta uses context parallelism only in its 131K-token stage. The match is narrow, since the winning split beats the runner-up (TP=8, PP=32) by only 1.4% in MFU. It does not depend on the optimizer's default cap of TP at the node size, because raising the cap to 64 returns the same TP=8, PP=16 split.

The cases exercise the Node, Algorithm, Fleet, and Operations domains, but the evidence differs across rows. The Data and Analysis domains have no independent performance comparison here. Table 6 preserves the distinction between performance error, identity, and an unavailable metric.

Registry entries record `metadata.provenance` (Section 3.1.1). Datasheet, literature, and industry-report numbers require a verified URL and date, estimates and derived values require documented justification, and the provenance audit tool (`mlsysim.tools.audit_provenance`) fails on any value in the core registries whose provenance is missing or weak, which keeps "magic numbers" from accumulating silently as the framework evolves.

6.2 Exploration Speed

On an Apple M5 Max laptop (Python 3.12.13), MLSYS·IM's `SingleNodeModel` evaluates 1,000 hardware–model–batch configurations in 266 ms (median of five warm runs), about 266 μs per configuration. This local runtime supports interactive sweeps of the configurations exercised here. It does not establish a speed ratio against a detailed simulator, which performs different work.

6.3 Accuracy Scope

MLSYS·IM provides first-order estimates rather than cycle-accurate simulations. The resolver abstraction grounds performance in first-principles physical bounds: memory traffic follows tensor footprints and bus bandwidths, collective communication scales with payload and topology, and pipeline bubbles follow exact scheduling equations with virtual stages v.

The principal assumptions relevant to these examples are listed below. This is not an exhaustive inventory of every heuristic in the package; solver documentation and `engine.calibration` record additional defaults.

1. **Single-device efficiency** (η): A declared accelerator-level efficiency representing silicon utilization, motivated by systolic tile quantization and ALU Amdahl's Law but supplied as an input (the examples use $\eta = 0.50$ for the Llama training comparison and $\eta = 0.19$ for ResNet-50).
2. **Communication–computation overlap** (η_{overlap}): A declared fraction ($0 \leq \eta_{\text{overlap}} \leq 1$, default 0.85) specifying the achievable degree of overlap between non-blocking collective communication and backward-pass compute.
3. **Framework and dispatch overhead**: Fixed per-device dispatch taxes (the H100 registry value is $T_{\text{dispatch}} = 10\,\mu$s) and a framework tax of 10 μs per layer. These are modeling defaults, not universal measured CUDA launch costs.
4. **Idle power fraction**: A floor on datacenter server power ($P_{\text{idle}} \approx 0.30$–$0.50\,P_{\text{peak}}$) reflecting dynamic power scaling limits.
5. **DP-SGD gradient clipping**: A scalar clipping threshold C supplied by the user when evaluating privacy-preserving training.

The distributed solver represents pipeline bubbles, communication, recomputation, and storage through explicit terms. Their accuracy depends on the selected scheduling and overlap approximations. Benchmarks 4–6 test accounting relationships; they do not supply independent evidence for a general latency or utilization error bound.

A first-order result is useful when it narrows the design question and states what remains uncertain. If two designs differ by less than plausible variation in bandwidth or efficiency, the model cannot reliably rank them. Sensitivity analysis should precede a procurement or deployment conclusion, and measurement should follow it.

Regression tests in `engine.empirical` cover calibrated ResNet throughput, a broad Llama decode range, and GPT-3 FLOP accounting. These categories are documented separately; the range is not a matched serving-engine experiment. The paper's validation script reports its comparisons and fails when a configured regression threshold, accounting identity, feasibility requirement, or search check fails.

6.4 Architectural Invariants

Beyond numerical accuracy, dimensional strictness (Section 3.1.2) provides structural correctness: unit mismatches raise immediate runtime errors rather than producing silently wrong numbers, eliminating a category of bugs that benchmark matching alone cannot detect.

Table 6: **Evaluation Summary.** Model outputs and reference values for seven evaluation cases. Error is $|(\text{pred.} - \text{rep.})/\text{rep.}|$. Benchmark 4 evaluates HFU-to-MFU accounting under rematerialization; Benchmarks 5 and 6 verify arithmetic against published scaling laws and emissions formulas. Model outputs are regenerated from the checked-out package. Case 2 compares different metrics and has no decode-error estimate.

Benchmark	Predicted	Reported	Error
1: ResNet-50 DGX A100	19,007 samples/s	17,400 images/s	9.2%
2: Llama-2 70B (TP=4)	10.45 ms weight-read floor	18.52 ms/output token, request average	n/a
3: Llama 3 405B MFU	40.6%	41%	1.0%
4: PaLM-540B HFU → MFU	41.1%	46.2%	11.0% (accounting)
5: Chinchilla P^*	70.0B	70B	identity
6: GPT-3 CO_2e	552.1 t	552 t	identity
7: Llama 3 parallelism	TP=8, PP=16, DP=128	TP=8, PP=16, DP=128	exact match

A complementary question is whether the framework's *architectural* invariants hold as the codebase evolves: do all hardware specifications flow through typed registries? Does every formula carry `pint` units end-to-end? Can a contributor accidentally reintroduce a bare constant that bypasses the registry? MLSYS·IM answers these with five machine-checked structural invariants, enforced by a test suite of 5,836 tests, each encoding a design decision that would otherwise erode through incremental changes:

1. **Specification separation.** There is no constants module at all: the legacy `core/constants.py` was deleted outright once its last value migrated, and a CI gate now pins its *absence* (any attempt to reintroduce the module, or a compatibility shim for it, fails). Every domain figure lives in its category registry; measurement units live in `core.units`.
2. **Loader contract.** The YAML registry loader rejects duplicate keys outright and requires every `@tech:` and `@prov:` reference to resolve, so a typo in a data file fails at import rather than silently shadowing or orphaning a specification.
3. **Provenance coverage.** A dedicated audit (`mlsysim.tools.audit_provenance`, also run as a test) walks every registry leaf and fails on missing or weak provenance, enforcing the kind-specific evidence rules of Section 3.1.1 across the core registries.
4. **Cross-registry consistency.** When the same physical quantity appears in multiple registries (e.g., NVLink bandwidth in both Hardware and Systems node definitions), a dedicated check verifies that the DGX node definitions agree with their accelerator entries. This prevents silent divergence between overlapping specification surfaces.
5. **Registry immutability.** Every registry entry is a frozen `pydantic` model: attribute assignment raises at the call site. Entries are shared across notebook cells, so rejecting direct field assignment prevents accidental replacement of specifications such as `H100.tdp`. Frozen models do not imply recursive immutability of every nested container. Code that needs a perturbed variant builds an explicit copy (`model_copy`).

Provenance and loader checks apply to every new registry entry automatically, while the cross-registry check covers the DGX node definitions. Published benchmark tests (`engine.empirical`) complete the picture by binding representative solver outputs to sourced benchmark envelopes. These checks protect the properties they exercise; they are not a proof that every future contribution preserves the architecture or the scientific model.

7 Usage & Case Studies

MLSYS·IM is designed for three audiences: students developing quantitative reasoning skills, instructors preparing demonstrations, and researchers evaluating design trade-offs. We present two representative use cases per persona, each illustrating how solvers compose to answer questions that span multiple walls. For a cookbook of runnable code examples demonstrating how users define hardware, perform algebraic design-space sweeps, and invoke SLA-driven hardware synthesis programmatically in Python, see the Appendix (Listings 8, 9, 10, 11, 12, 13, 14, 15, and 16).

7.1 Student Use Cases

Students interact with MLSYS·IM primarily through single-solver queries, short resolver chains, and browser-friendly exercises in the companion textbook. They

can manipulate hardware parameters (e.g., batch size, SLA targets, carbon taxes) and observe how binding constraints shift without needing a backend server or physical hardware.

We present two examples chosen to illustrate complementary aspects of the framework. The first (S1) separates generated token volume from request latency in a chain-of-thought scenario. The second (S2) demonstrates *horizontal* composition, combining data-pipeline and compute models to diagnose a bottleneck that moves from the accelerators to the CPU input pipeline.

7.1.1 S1: Chain-of-Thought Cost

A student investigates how chain-of-thought prompting changes inference work. They run Llama-3 70B on a B200, where the FP16 model fits, and compare one reasoning step with $K = 8$ steps using `InferenceScalingModel`. Under the solver's assumed token budget per step, total time is $T_{\text{reason}} = \text{TTFT} + K\,N_{\text{step}}\,\text{ITL}$. The modeled times are 1.19 s and 7.7 s, a latency ratio of 6.4. Generated token volume grows by a factor of 8; latency grows less because the prefill occurs once. These results describe a fixed token-budget scenario, not a prediction of answer quality. Converting token volume into fleet cost requires a separate serving-capacity calculation that accounts for batching, concurrency, and the latency target.

7.1.2 S2: CPU Pipeline Bottleneck

A student configures ResNet-50 training on a DGX A100 (8 GPUs) with a batch size of 2,048. At a kernel-level efficiency of $\eta = 0.30$, the `DistributedModel` evaluated on a single DGX A100 node predicts a step time of ∼69 ms, a demand rate of ∼29,637 images/s. Adding the `DataModel` rules out storage I/O. With 32 CPU workers (4 per GPU) decoding ImageNet JPEGs at 1,200 images/s each, the raw decode pipeline delivers 38,400 images/s. This appears sufficient, but the `TransformationModel` accounts for the full augmentation pipeline (random crop, color jitter, normalization) at 850 images/s per worker, reducing effective throughput to 27,200 images/s, below the GPUs' 29,637 images/s appetite. The student discovers that the binding constraint shifts from the accelerators to Wall 9 (Transformation). The GPUs have spare cycles, accelerator utilization falls to 92%, and the achieved end-to-end efficiency drops to $\eta \approx 0.28$, demonstrating how MLSYS·IM isolates the upstream data pipeline bottleneck rather than falsely attributing the throughput loss to accelerator execution.

7.2 Instructor Use Cases

Instructors need demonstrations that run in real time, produce concrete numbers, and connect cleanly to lecture narratives. The following cases show how MLSYS·IM turns abstract concepts into live, interactive classroom demonstrations.

7.2.1 I1: Live Roofline Demo, Batch Size Sweep

An instructor demonstrates the Roofline model by sweeping batch size from 1 to 256 on an H100 for ResNet-50 inference at $\eta = 0.5$. At each batch size, the `SingleNodeModel` returns the bottleneck label, the arithmetic intensity, and the MFU:

Batch	Bottleneck	AI (FLOP/B)	MFU
1	Memory	146	0.02
8	Compute	713	0.10
32	Compute	1,222	0.25
128	Compute	1,488	0.40
256	Compute	1,544	0.45

The crossover occurs between batch 1 and batch 8, near the effective ridge point $\eta F_{\text{peak}}/BW_{\text{HBM}} = 148$ FLOP/byte. Weight reads amortize across samples, while the generic engine's per-sample activation-traffic approximation limits further gains. The contrast sweep for a 7B transformer saturates near 9.6 FLOP/byte under that approximation. It is not a KV-aware serving result and does not imply that decode remains memory bound at every batch size. `ServingModel` should be used for the compute/KV comparison in Equation (7). The five-point sweep takes about 1.31 ms on the evaluation machine.

7.2.2 I2: Iowa vs. Québec Carbon

An instructor poses a policy question: *does geography matter for carbon footprint?* They configure a 256-GPU cluster (`Systems.Clusters.Research_256`) running a training job for 30 days at MFU 0.42, and the `SustainabilityModel` computes emissions under two registered datacenter profiles: an illustrative high-carbon reference profile labeled Iowa (680 gCO_2/kWh, PUE 1.12) and Québec (hydroelectric, 20 gCO_2/kWh, PUE 1.06). The configuration is identical in hardware and MFU, yet the run emits 58.4 tonnes CO_2 under the Iowa profile versus 1.6 tonnes in Québec, a 36× gap, demonstrating that grid carbon intensity is a first-order systems design variable (Wall 18). Figure 7 visualizes this contrast.

7.3 Researcher Use Cases

Researchers need to evaluate architectural alternatives and justify procurement decisions with quantitative evidence. The following cases show how MLSYS·IM enables rapid what-if analysis across hardware generations and parallelism strategies.

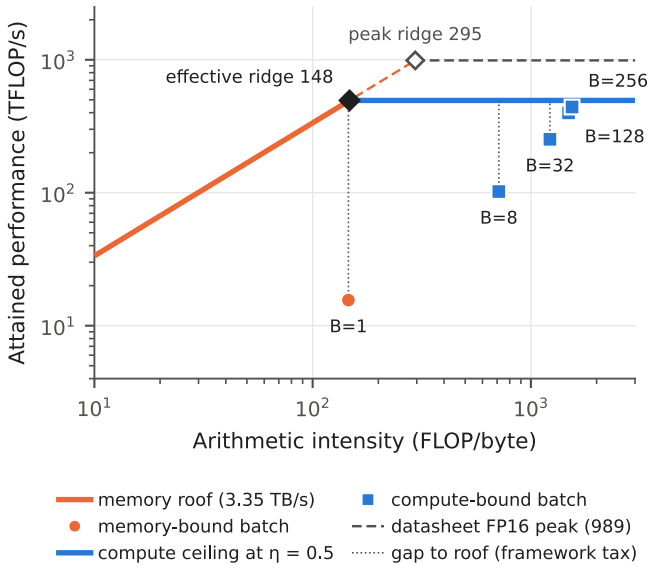


Figure 6: **Roofline Crossover: ResNet-50 Batch Size Sweep on H100.** Increasing batch size moves the operating point rightward along the Roofline, transitioning from memory-bound (purple) to compute-bound (cyan) between batch 1 and batch 8. The peak ridge point is $989/3.35 \approx 295$ FLOP/byte; at $\eta = 0.5$ the effective ridge sits at ≈148 FLOP/byte. MFU continues rising past the crossover as dispatch and framework overheads amortize.

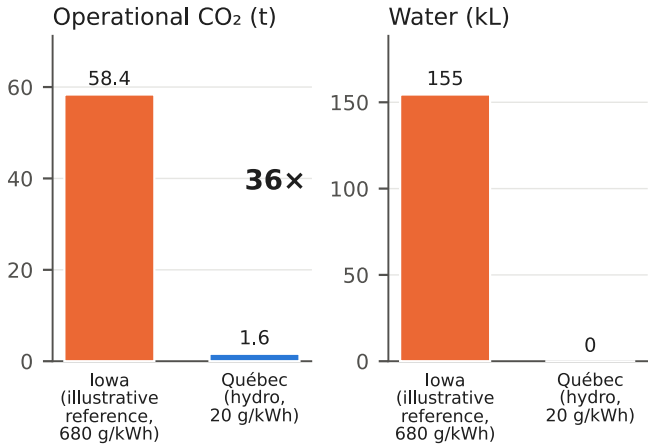


Figure 7: **Geography as a Systems Variable: Iowa vs. Québec.** An identical 256-GPU cluster running for 30 days at MFU 0.42 emits 36× more CO_2 under the illustrative Iowa reference profile (680 gCO_2/kWh, PUE 1.12) than in Québec (hydroelectric, 20 gCO_2/kWh, PUE 1.06), 58.4 vs. 1.6 tonnes. Hardware and MFU are identical; only the regional grid profile differs between the two sites.

7.3.1 R1: GPU vs. Cerebras Crossover

A researcher evaluates whether wafer-scale silicon changes the economics of large-model inference, using a user-defined 180B-parameter, 80-layer transformer (the registry has no model of this size). On H100s, the 360 GB FP16 checkpoint requires tensor parallelism across 5 GPUs, and MLSys·IM computes a weight-read lower bound of $360\ \text{GB}/(5 \times 3.35\ \text{TB/s}) \approx 21.5$ ms/token at batch 1. This bound excludes KV-cache traffic, communication, and execution overhead; it is not a serving-latency prediction. The `WeightStreamingModel` then models the Cerebras WSE-3 with a MemoryX injection bandwidth of 1.2 TB/s (Cerebras has not published an official MemoryX bandwidth figure, so this value is an estimate). At batch size 1, the weights must stream from MemoryX at this rate, yielding ∼300 ms per token, a weight-streaming term larger than the GPU weight-read floor under these bandwidth assumptions. This comparison does not establish end-to-end serving performance. Equation (15) places the optimal batch at $B^* \approx 41{,}666$ for $\eta = 0.4$, where injection cost would fully amortize, but at that batch the per-request KV-cache and activation state exceed the 44 GiB wafer SRAM by roughly 326×, and the solver marks the configuration infeasible. The largest feasible batch is 127, where the wafer delivers ∼423 tokens/s. The binding constraint therefore shifts from BW_{inject} at small batches to on-wafer memory at large ones, illustrating that the optimal architecture depends on batch size and on where request state can live.

7.3.2 R2: Hardware Procurement Audit

A researcher preparing a hardware procurement recommendation for LLM inference needs to answer: *should the next-generation cluster prioritize FLOPS or bandwidth?* The `SensitivitySolver` perturbs each hardware parameter by 10% and reports the normalized latency response, shown here for a single-GPU-resident Llama-3 8B decode workload:

```python
import mlsysim
from mlsysim.solvers import SensitivitySolver
solver = SensitivitySolver()
res = solver.solve(model=mlsysim.Models.Language.Llama3_8B,
                   hardware=mlsysim.Hardware.Cloud.A100)
print(res.sensitivities)
# {'peak_flops': 0.0, 'memory_bandwidth': -0.088,
#  'memory_capacity': 0.0}
print(f"Binding: {res.binding_constraint}")
# Output: Binding: memory_bandwidth
```

Listing 4: **Sensitivity Analysis.** Identifying the binding constraint via 10% hardware perturbations.

The result is unambiguous. A 10% increase in HBM bandwidth cuts decode latency by 8.8%, whereas a 10%

increase in peak FLOPS yields nothing at all, because batch-1 decode sits strictly on the memory roof of Equation (3). The `SynthesisSolver` then performs the inverse solve. Given a 50 ms inter-token latency SLA for Llama-3 70B, it derives the minimum required bandwidth as $\text{BW}_{\text{req}} = |W|/T_{\text{target}} \approx 2{,}824\,\text{GB/s}$, 1.38× the A100's 2.04 TB/s, confirming that Llama-3 70B inference is firmly in the memory-bound regime and that hardware procurement should prioritize HBM bandwidth over peak throughput.

7.3.3 R3: End-to-End Llama-3 70B Training Audit

To illustrate how MLSYS·IM's solvers compose across the taxonomy, we trace a training analysis for Llama-3 70B on the registered `Training_512_H100` fleet (64 DGX H100 nodes and 512 GPUs, NVLink intra-node and 200 Gb/s IB HDR inter-node) with a global batch of 1,024 sequences of 4,096 tokens, TP=8 within each node, DP=64, and $\eta = 0.40$.

Node (Walls 1–2). Without optimizer sharding or recomputation, the TP=8 training state exceeds an 80 GB H100, so the audit adds ZeRO-1 optimizer sharding across the DP group and full activation recomputation. The `DistributedModel` then evaluates one tensor-parallel replica with the Roofline core and predicts 11.70 s of compute per step, and the `TrainingMemoryModel` reports 48.6 GB of training state per GPU, which fits the HBM. Recomputation adds one forward-pass equivalent to the usual forward-plus-backward work, so the replica's model FLOPs utilization is 30.0% at $\eta = 0.40$.

Algorithm (Wall 11). A 30-day run at the predicted step time processes ∼0.82T tokens, a budget of $C \approx 3.48 \times 10^{23}$ FLOPs, for which the `ScalingModel` gives a Chinchilla-optimal model size of $P^* \approx 54$B.

Fleet (Walls 14–15). Communication adds 1,338 ms of tensor-parallel AllReduce and 1,391 ms of data-parallel gradient synchronization per step, of which 209 ms is exposed after compute–communication overlap, for a step time of 13.25 s, a scaling efficiency of 88.3%, and a fleet MFU of 26.5%. The `ReliabilityModel` estimates a fleet MTBF of 73.2 hours, a Young–Daly checkpoint interval of ∼94 minutes, about 10 failures over 30 days, and a goodput of 98.8%.

Operations (Walls 17–18). In Québec, the `EconomicsModel` projects $351K of amortized CapEx, $15.9K of energy, and $53K of maintenance, a run TCO of ∼$0.42M. The `SustainabilityModel` estimates 133 MWh of facility energy and 2.7 tonnes CO_2, 36× less than the 95.4 tonnes under the Iowa reference profile.

Analysis (Walls 21–22). The `SensitivitySolver` and `SynthesisSolver` operate on a single device, so a fleet-level what-if instead re-runs the chain with a changed fabric, node count, or region, which takes ∼0.001 s per evaluation on the recorded machine.

This trace exercises Walls 1, 2, 11, 14, 15, 17, and 18 through a single model, demonstrating how solver composition produces a system assessment from individual physics-based constraint equations.

7.3.4 R4: Automated Parallelism Search

A researcher needs to schedule a 175B-parameter model on a new 2,048-GPU cluster. Manually searching the 3D-parallelism space (TP × PP × DP) is error-prone. A split that maximizes DP might exceed the 80 GB HBM capacity, while a split that maximizes TP might saturate the NVLink interconnect. Instead of trial and error, the researcher invokes a Tier 3 Optimizer. They configure the `ParallelismOptimizer` with the workload, the cluster, a global batch of 2,048 sequences, and $\eta = 0.5$. It maximizes MFU over power-of-two factorizations with TP capped at the node size. A quick screen drops splits whose per-GPU weights, gradients, and Adam optimizer state exceed 90% of HBM (77.3 GB); the `DistributedModel` then prices each survivor with one-sequence pipeline microbatches and drops any whose training state, activations included, exceeds the 85.9 GB of an H100, leaving 13 candidates. In well under a second, it returns TP=8, PP=8, DP=32 (MFU 39.6%), with TP matching the intra-node GPU count so tensor-parallel traffic stays off the slower inter-node fabric. PP=8 is the shallowest pipeline whose per-GPU training state fits (55.1 GB); halving the pipeline depth would demand 98.7 GB. This demonstrates the power of the "engineering engine" to invert the analytical models into automated design-space synthesis.

8 Fallacies & Pitfalls

Following the tradition of Hennessy et al. (2024), we highlight common misconceptions that MLSYS·IM is designed to expose.

Fallacy: Tripling peak FLOP/s triples LLM serving speed. A student might assume that upgrading from A100 (312 TFLOP/s FP16) to H100 (989 TFLOP/s, a 3.2× increase) should yield a 3.2× speedup. MLSYS·IM's `SingleNodeModel` reveals why this is false. At small batch sizes, weight-bound decode depends on memory bandwidth, and the A100-to-H100 bandwidth improvement is only about 1.6× (2.04 TB/s → 3.35 TB/s). For memory-bound decode, token latency scales with bandwidth, not FLOP/s; compute-bound training does see most of the FLOP/s gain. The Roofline model (Equation (3)) makes this visible by showing that the binding constraint determines which hardware parameter matters.

Fallacy: At 512 GPUs, checkpointing is a bigger tax than communication. On the 200 Gb/s IB HDR fleet of Case R3, the `DistributedModel` charges 1,338 ms of tensor-parallel and 209 ms of exposed data-parallel communication to a 13.25 s step, a 11.7% loss of scaling efficiency. The `ReliabilityModel`, with a fleet MTBF of 73.2 hours and Young–Daly checkpoints every ∼94 minutes, charges only 1.2% of goodput to checkpoints and restarts. Even adding the expected rework term ($\tau_{\text{opt}}/(2 \cdot \text{MTBF}) \approx \delta/\tau_{\text{opt}}$ under Young–Daly, roughly doubling the loss to 2.4%), communication (Wall 14 at 11.7%) remains roughly five times larger at this scale. An analysis that blames checkpointing without measuring both would invest in the wrong fix.

Pitfall: Using peak bandwidth in back-of-the-envelope calculations. Vendor datasheets report peak HBM bandwidth (e.g., 3.35 TB/s for H100), and sustained bandwidth under real access patterns can be lower, so decode estimates built on peak bandwidth are optimistic. MLSYS·IM applies η only to FLOP/s; decode latency uses the registry bandwidth directly, so users who want a sustained-bandwidth estimate should derate `memory.bandwidth` explicitly (e.g., through `model_copy`).

Pitfall: Ignoring geography in carbon accounting. Two identical training runs produce vastly different environmental impact depending on grid carbon intensity. As demonstrated in Case I2, the same 256-GPU cluster emits 36× more CO_2 under the Iowa reference profile than in Québec. Students who omit Wall 18 (Sustainability) from their analysis miss a first-order systems design variable, one that increasingly affects both cost (carbon pricing) and regulatory compliance.

Fallacy: Quantization always provides a linear speedup. Reducing precision from FP16 to INT4 ($r_{\text{quant}} = 4\times$) reduces memory by 4×, but the compute speedup depends on whether the workload is memory-bound or compute-bound. For memory-bound LLM decode, most of the 4× memory reduction becomes throughput because the bottleneck is weight reads. For compute-bound workloads, the gain depends on low-precision kernel support, so MLSYS·IM applies the registry's per-precision FLOP/s where an entry exists and falls back to FP16 throughput, with a warning, when it does not. MLSYS·IM's Roofline analysis makes this regime-dependent behavior explicit.

9 Discussion & Limitations

The model omits execution details that can alter both latency and the apparent bottleneck. The following limits determine when its results can guide an experiment and when a more detailed model is needed.

9.1 What MLSYS·IM Cannot Model

No microarchitectural cycle simulation. The model does not simulate cache replacement, warp scheduling, register spills, or instruction pipelines. A declared efficiency scales compute throughput, but it cannot identify which of these mechanisms causes a loss. Applying an efficiency measured in one kernel or workload to another requires separate justification.

No real network congestion. The communication model uses the classical α-BW_{link} formulation (latency plus inverse-bandwidth), which assumes dedicated links. MLSYS·IM does not model adaptive routing, network contention under multi-tenant traffic, or congestion collapse, effects whose importance depends on traffic and topology. A detailed network simulator such as ASTRA-sim 2.0 (Won et al., 2023) can examine those interactions. The static bisection view behind Wall 10 (Locality) is optimistic under the same cross-traffic conditions. A scalar `congestion_factor` can inflate communication time, but contention dynamics are not modeled.

Coarse OS/runtime overhead. Kernel launch and framework dispatch are folded into a fixed per-device dispatch tax and a per-layer framework tax; CUDA stream scheduling, Python GIL contention, and host–device transfer dynamics are not modeled separately, so small-batch inference predictions are only as good as those two constants.

No heterogeneous fleets. The `DistributedModel` assumes homogeneous nodes, where all accelerators in a fleet share the same compute, memory, and interconnect specifications. Production clusters increasingly mix hardware generations (e.g., A100 and H100 nodes in the same job), and fleet-level efficiency metrics such as ML Productivity Goodput (Wongpanich et al., 2025) capture this heterogeneity. Modeling heterogeneous fleets would require per-node load balancing and straggler analysis beyond the current analytical framework.

No dynamic behavior. MLSYS·IM models steady-state throughput. Transient effects (thermal throttling, dynamic clock boosting, memory fragmentation over long training runs, and checkpoint I/O contention with concurrent training traffic) are outside its scope. A training run that degrades over 72 hours due to thermal saturation will appear identical to one that sustains peak throughput.

Heuristic accuracy models. The accuracy degradation curves in `CompressionModel` are heuristic step functions (for quantization) and exponentials (for pruning), not architecture-specific empirical fits. Real accuracy loss depends on model architecture, calibration methodology, and quantization method (e.g., well-calibrated GPTQ INT4 can achieve <0.5% degradation, while naive round-

to-nearest may lose 10%+). Users should treat these curves as directional indicators, not ground truth.

9.2 Walls Not Included

The 22-wall taxonomy is a selection of analytically tractable constraints. Several constraints were considered and excluded, each for a specific reason.

Thermal throttling. Sustained thermal saturation can dynamically force accelerator clocks below base frequencies over multi-hour training runs. MLSYS·IM models steady-state datasheet clock frequencies and peak throughput, leaving transient dynamic thermal throttling outside its analytical scope.

Resource fragmentation. Scattered GPU availability across nodes prevents job scheduling even when aggregate capacity is sufficient; this is a combinatorial bin-packing problem beyond the current analytical framework.

Compiler/graph optimization. The gap between a framework's computational graph and the executed kernel schedule affects both latency and MFU, but varies too rapidly across software versions to model analytically.

The selection criterion for inclusion was: does the constraint have a stable, published analytical formulation that remains valid across hardware generations? Constraints requiring empirical trace data or combinatorial optimization were deferred to future work.

9.3 The Accuracy–Speed Trade-off

Each omission above reflects the same trade-off: evaluation that is orders of magnitude faster, at the cost of second-order fidelity. The precedent is SPIM, which executes MIPS programs at the instruction-set level and omits pipelining, superscalar execution, and cache hierarchies, prioritizing pedagogical clarity over the full complexity of commercial processors. MLSYS·IM applies the same philosophy to ML systems, making quantitative reasoning accessible to students who may never operate a production cluster.

Bottleneck identification and numerical accuracy need separate evaluation. A clear gap between compute and memory time can support a useful hypothesis about which resource limits execution. Near the crossover, a modest change in sustained bandwidth or efficiency can reverse that conclusion. The current comparisons do not measure bottleneck-classification accuracy across a held-out set, so the framework's labels should be tested through sensitivity analysis and measurements of the target system.

9.4 Future Work

We identify several directions for extending MLSYS·IM.

Broader empirical evaluation. Further evaluation should use matched workload, topology, runtime, and metric definitions, with efficiency assumptions declared before comparing outputs. The Argonne request timings illustrate why a published latency is not automatically a decode-ITL reference. Useful additions include held-out serving traces, sustained-bandwidth measurements, bottleneck-label comparisons, and runs on newer devices such as B200/GB200 and TPU v6. These comparisons should report failures and uncertainty as well as agreement.

Community hardware registry. The Hardware registry currently contains a curated set of hardware entries sourced to manufacturer datasheets where available, with estimates labeled as such in their provenance. We plan to open contributions from the community, with automated verification scripts that cross-check submitted specifications against known physical limits (e.g., memory bandwidth cannot exceed pin count $\times$ data rate).

Custom degradation curves. Future versions will allow users to supply empirically fitted accuracy-compression curves from their own quantization experiments, replacing the current heuristic step and exponential curves with data-grounded models.

TinyTorch integration. MLSYS·IM provides analytical predictions; TinyTorch (Reddi et al., 2025), the companion educational framework, provides implementation-based verification. Connecting the two tools creates a predict-then-verify loop. Students estimate training time and memory consumption in MLSYS·IM, then run the actual training in TinyTorch and compare. This closed loop reinforces quantitative reasoning by grounding analytical models in empirical observation.

Agentic and embodied extensions. The package already ships 4 extension registries, Agents (agentic workload profiles), Embodied (robot platforms), Sensors, and Actuators, together with the `agents`, `robotics`, and `drivetrain` physics modules (Table 4). They cover trajectory reliability, test-time compute cost, and multi-agent coordination for agents, and sensor-to-actuator latency, safe stopping distance, action-chunk cadence, and braking limits for robots. We deliberately keep them outside the wall taxonomy. Several of their formulas are modeling assumptions rather than published analytical results, many registry values are labeled estimates, but this paper does not evaluate their domain solvers or reference benchmarks. Promoting candidates such as a trajectory-reliability wall or a sensor-to-actuator latency wall would require sourced formulations, dedicated resolvers, and published benchmarks of the kind in Section 6.

Expanding Tier 3 Optimizers (Pareto Frontiers). Currently, the Tier 3 optimizers search single-dimensional objective spaces (e.g., maximizing MFU or maximizing batch size under a latency constraint); a two-objective compression Pareto sweep already ships in `CompressionModel.sweep`. Future work will extend the Tier 3 engine to support multi-objective Pareto frontiers, simultaneously optimizing across latency, total cost, carbon footprint, and accuracy. This will enable richer design-space exploration and formally expose the inherent tensions between performance and sustainability.

9.5 The Pedagogical Argument

The educational value lies in making reasoning available for inspection. A student can state a workload, compute its resource demands, vary an assumption, and explain why the limiting term changes. The worked examples demonstrate this workflow, but they do not measure learning gains. Establishing those gains would require a study that compares student reasoning before and after using the tool. Likewise, the Llama training comparison depends on declared efficiency and schedule assumptions; its agreement with one published result does not validate the teaching method or eliminate the need for further performance measurements.

10 Conclusion

Compute throughput, memory traffic, communication, and operating constraints jointly determine whether an ML configuration is useful. MLSYS·IM expresses selected relationships among these quantities as executable models. Its registries supply named inputs, its unit system checks dimensional compatibility, and its resolvers expose the terms that determine an estimate.

The framework supports the stage before detailed measurement: checking whether a configuration fits, estimating which resource may limit it, and selecting alternatives worth testing. The evaluation shows reproducible accounting and selected comparisons with published performance. It also exposes missing execution details and mismatched metrics. Those limitations define the scope of the result rather than exceptions to a general accuracy claim.

As the modeling engine behind the textbook series, MLSYS·IM connects an equation to a runnable calculation without requiring a GPU cluster. The intended habit is simple: state the assumptions, keep the units, change one input, and explain the result. Measurements and more detailed simulators remain necessary to determine how well that calculation describes a particular implementation.

Generative AI Usage Statement

In developing MLSYS·IM, the goal was not to formulate new physical laws or mathematical bounds, but to ground ML systems reasoning in first principles—distilling established equations and published constraints from across computer architecture, distributed systems, and machine learning into an executable, dimensionally strict engineering framework. The package was designed specifically as the quantitative substrate for the *Machine Learning Systems* textbook series (Janapa Reddi, 2025), providing an environment for first-principles architectural reasoning without requiring hardware clusters. Generative AI tools were used throughout this process to assist with cross-disciplinary literature search, locate empirical data points, and draft initial code and test scaffolding from structural blueprints. I conceived the system architecture, structured the 22-wall taxonomy and progression, designed the software interfaces, and reviewed the implementation and its source attributions. Automated dimensional and regression tests check selected properties; they do not prove every formula or source claim. I remain responsible for the design, implementation, and conclusions presented in this work.

References

Martin Abadi, Andy Chu, Ian Goodfellow, et al. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 308–318. ACM, 2016. doi: 10.1145/2976749.2978318. URL https://doi.org/10.1145/2976749.2978318.

A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee. Efficient LLM inference via chunked prefills. *ACM SIGOPS Operating Systems Review*, 59(1):9–16, None 2025. ISSN 0163-5980. doi: 10.1145/3759441.3759444. URL https://doi.org/10.1145/3759441.3759444.

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2024. URL https://arxiv.org/abs/2405.05465.

Abhimanyu Bambhaniya, Ritik Raj, Geonhwa Jeong, Souvik Kundu, Sudarshan Srinivasan, Midhilesh Elavazhagan, Madhu Kumar, and Tushar Krishna. Demystifying platform requirements for diverse LLM

inference use cases. *arXiv preprint arXiv:2406.01698*, 2024.

Luiz André Barroso and Urs Hölzle. The case for energy-proportional computing. *Computer*, 40(12):33–37, 2007. ISSN 0018-9162. doi: 10.1109/mc.2007.443. URL https://doi.org/10.1109/mc.2007.443.

Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer*. Synthesis Lectures on Computer Architecture. Springer International Publishing, 2019. ISBN 9783031006333, 9783031017612. doi: 10.1007/978-3-031-01761-2. URL https://doi.org/10.1007/978-3-031-01761-2.

Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011. ISSN 0163-5964. doi: 10.1145/2024716.2024718. URL https://doi.org/10.1145/2024716.2024718.

Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffenetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. Llm-inference-bench: Inference benchmarking of large language models on ai accelerators. *arXiv preprint arXiv:2411.00136*, 2024. URL https://arxiv.org/abs/2411.00136.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL http://arxiv.org/abs/2204.02311v5.

Benoit Courty, Victor Schmidt, et al. mlco2/codecarbon: v2.4.1. Zenodo, 2024. URL https://doi.org/10.5281/zenodo.11171501.

Russ Cox, M. Frans Kaashoek, and Robert Morris. xv6: A simple, Unix-like teaching operating system. MIT PDOS, 2011. URL https://pdos.csail.mit.edu/6.828/xv6.

J.T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006. ISSN 0167-739X. doi: 10.1016/j.future.2004.11.016. URL https://doi.org/10.1016/j.future.2004.11.016.

Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013. ISSN 0001-0782, 1557-7317. doi: 10.1145/2408776.2408794. URL https://doi.org/10.1145/2408776.2408794.

DeepSeek-AI. Insights into DeepSeek-V3: Scaling challenges and reflections on hardware for AI architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 1731–1745. ACM, 2025. doi: 10.1145/3695053.3731412. URL https://doi.org/10.1145/3695053.3731412.

Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2022.

Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. LLMCarbon: Modeling the end-to-end carbon footprint of large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization

for generative pre-trained transformers, 2023. URL https://doi.org/10.48550/ARXIV.2210.17323.

Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision: Improve the Efficiency of Artificial Intelligence*, pages 291–326. Chapman and Hall/CRC, 2022. ISBN 9781003162810. doi: 10.1201/9781003162810-13. URL https://doi.org/10.1201/9781003162810-13.

Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S. Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. Chasing carbon: The elusive environmental footprint of computing. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 854–867. IEEE, 2021. doi: 10.1109/HPCA51647.2021.00076. URL https://doi.org/10.1109/HPCA51647.2021.00076.

Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint*, 2016. URL http://arxiv.org/abs/1510.00149v5.

Mor Harchol-Balter. *Performance Modeling and Design of Computer Systems*. Cambridge University Press, 2013. doi: 10.1017/CBO9781139226424. URL https://doi.org/10.1017/CBO9781139226424.

John L. Hennessy, David A. Patterson, and Christos Kozyrakis. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 7th edition, 2024. ISBN 978-0443154065.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030. Curran Associates Inc., 2022. doi: 10.52202/068431-2176. URL https://doi.org/10.52202/068431-2176.

Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. Calculon: A methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14. ACM, 2023. doi: 10.1145/3581784.3607102. URL https://doi.org/10.1145/3581784.3607102.

Vijay Janapa Reddi. *Machine Learning Systems: Design and Implementation*. MIT Press, 2025. URL https://mlsysbook.ai. Forthcoming. Early access at <https://mlsysbook.ai>.

Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2019. URL https://arxiv.org/abs/1807.05358.

John Kim, William J. Dally, Steve Scott, and Dennis Abts. Technology-driven, highly-scalable dragonfly topology. In *International Symposium on Computer Architecture (ISCA)*, pages 77–88. IEEE, 2008. doi: 10.1109/ISCA.2008.19. URL https://doi.org/10.1109/ISCA.2008.19.

Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Charles E. Leiserson. Fat-trees: Universal networks for hardware-efficient supercomputing. *IEEE Transactions on Computers*, C-34(10):892–901, 1985. ISSN 0018-9340. doi: 10.1109/tc.1985.6312192. URL https://doi.org/10.1109/tc.1985.6312192.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020. URL https://arxiv.org/abs/2006.16668.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.

In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Cheng Li. llm-analysis: Latency and memory analysis of transformer models for training and inference. https://github.com/cli99/llm-analysis, 2023. Accessed: 2025-01-15.

Mingyu Liang, Hiwot Tadese Kassa, Wenyin Fu, Brian Coutinho, Louis Feng, and Christina Delimitrou. Lumos: Efficient performance modeling and estimation for large-scale LLM training. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2025.

Sean Lie. Cerebras architecture deep dive: First look inside the HW/SW co-design for deep learning. In *IEEE Hot Chips 34 Symposium*. IEEE, 2022.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Guangxuan Xiao, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *GetMobile: Mobile Computing and Communications*, 28(4):12–17, 2024. ISSN 2375-0529, 2375-0537. doi: 10.1145/3714983.3714987. URL https://doi.org/10.1145/3714983.3714987.

Llama Team, AI Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. Analyzing and mitigating data stalls in DNN training. *Proceedings of the VLDB Endowment*, 14(5):771–784, 2021. ISSN 2150-8097. doi: 10.14778/3446095.3446100. URL https://doi.org/10.14778/3446095.3446100.

Derek G. Murray, Jiri Simsa, Ana Klimovic, and Ihor Indyk. tf.data: A machine learning data processing framework. *Proceedings of the VLDB Endowment*, 14 (12):2945–2958, 2021.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. ACM, 2021. doi: 10.1145/3458817.3476209. URL https://doi.org/10.1145/3458817.3476209.

NVIDIA Corporation. ResNet-50 v1.5 for TensorFlow: Training performance on NVIDIA DGX A100. NVIDIA DeepLearningExamples repository, 2020. URL https://github.com/NVIDIA/DeepLearningExamples/tree/master/TensorFlow/Classification/ConvNets/resnet50v1.5. TensorFlow 20.06-tf1-py3 NGC container, 8x A100 40GB, batch size 256 per GPU, mixed precision with XLA. Accessed: 2026-09-15.

NVIDIA Corporation. NVIDIA H100 tensor core GPU: Product specifications. https://www.nvidia.com/en-us/data-center/h100/, 2023. Accessed: 2026-09-15.

Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315. IEEE, 2019. doi: 10.1109/ispass.2019.00042. URL https://doi.org/10.1109/ispass.2019.00042.

Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009. doi: 10.1016/j.jpdc.2008.09.002. URL https://doi.org/10.1016/j.jpdc.2008.09.002.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024. doi: 10.1109/isca59077.2024.00019. URL https://doi.org/10.1109/isca59077.2024.00019.

David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021. URL https://arxiv.org/abs/2104.10350.

David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2014. ISBN 978-0124077263.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5. mlsys.org, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023

/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.

Hang Qi, Evan R. Sparks, and Ameet Talwalkar. PALEO: A performance model for deep neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020. doi: 10.1109/sc41405.2020.00024. URL https://doi.org/10.1109/sc41405.2020.00024.

Vijay Janapa Reddi et al. TinyTorch: A progressive educational framework for machine learning systems. Harvard University, 2025. URL https://mlsysbook.ai/tinytorch.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. URL http://arxiv.org/abs/1909.08053v4.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025.

Arthur G. Stephenson, Lia S. LaPiana, Daniel R. Mulville, Peter J. Rutledge, Frank H. Bauer, David Folta, Greg A. Dukeman, Robert Sackheim, and Peter Norvig. Mars Climate Orbiter Mishap Investigation Board Phase I Report. Technical report, National Aeronautics and Space Administration, 1999.

Richard S. Sutton. The bitter lesson. *incompleteideas.net*, 43, 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html. Blog post on the importance of general methods and computation over specialized algorithms.

Andrew S. Tanenbaum and Albert S. Woodhull. *Operating Systems: Design and Implementation*. Prentice Hall, 3rd edition, 2006.

Xizheng Wang et al. SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2025.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline. *Communications of the ACM*, 52 (4):65–76, 2009. ISSN 0001-0782, 1557-7317. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 283–294. IEEE, 2023. doi: 10.1109/ispass57527.2023.00035. URL https://doi.org/10.1109/ispass57527.2023.00035.

A. Wongpanich, T. Oguntebi, J. Baiocchi Paredes, Y. E. Wang, P. M. Phothilimthana, R. Mitra, Z. Zhou, N. Kumar, and V. J. Reddi. Machine learning fleet efficiency: Analyzing and optimizing large-scale Google TPU systems with ML productivity goodput, 2025. URL https://arxiv.org/abs/2502.06982.

Bichen Wu, Kurt Keutzer, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, and Yangqing Jia. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10726–10734. IEEE, 2019. doi: 10.1109/cvpr.2019.01099. URL https://doi.org/10.1109/cvpr.2019.01099.

Wm. A. Wulf and Sally A. McKee. Hitting the memory wall. *ACM SIGARCH Computer Architecture News*, 23 (1):20–24, 1995. doi: 10.1145/216585.216588. URL https://doi.org/10.1145/216585.216588.

John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974. ISSN 0001-0782, 1557-7317. doi: 10.1145/361147.361115. URL https://doi.org/10.1145/361147.361115.

Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*. USENIX Association, 2021.

Zhihang Yuan, Yuzhang Shang, Yang Zhou, et al. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024.

Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. LLMCompass: Enabling efficient hardware design for large language model inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1080–1096. IEEE, 2024. doi: 10.1109/isca59077.2024.00082. URL https://doi.org/10.1109/isca59077.2024.00082.

Lianmin Zheng, Liangsheng Ying, Zhenghao Yin, Cody Hao Liu, Eric Yu, Prakhar Patel, Zhipeng Yang, Anthony K. Joseph, and Ion Stoica. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024. doi: 10.48550/arXiv.2312.07104. URL https://arxiv.org/abs/2312.07104.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, jul 2024. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

Appendix A MLSysBook Integration (LEGO Cells)

MLSys·IM serves as the executable backbone for *Machine Learning Systems*, a four-volume series (https://mlsysbook.ai/vol1, https://mlsysbook.ai/vol2, https://mlsysbook.ai/vol3, and https://mlsysbook.ai/vol4) (Janapa Reddi, 2025). To integrate quantitative analysis into the prose without disrupting readability, the textbook introduces the concept of LEGO Cells. LEGO is an acronym for the four structural phases of each cell: Load (importing registries and constraints), Execute (invoking the physics solvers), Guard (asserting dimensional and logical invariants), and Output (formatting physical quantities for inline prose). A LEGO Cell is a standalone, self-contained Python snippet that answers a specific design question posed in the text. Isolating the simulation logic into these executable blocks allows readers to read the code to understand the mechanics, execute it to verify the results, and modify the parameters to explore alternative scenarios.

Below, we provide three illustrative cells written in the LEGO style used throughout the textbook. They are simplified for this paper and are not verbatim excerpts; the cells in the books wrap the same four phases in a class with a documented header.

A.1 Volume 1 Style: Hardware Constraints

In Volume 1, LEGO Cells are heavily used to explain hardware constraints. For example, when introducing the Roofline model, a LEGO-style cell sweeps Llama-3 8B on an H100 using the generic engine's activation-traffic approximation. This is a Roofline exercise; KV-aware decode requires the serving solver shown below:

Listing 5: **Illustrative LEGO-Style Cell: Roofline Analysis.** Sweeping batch size with the `Engine` shows the memory-bound regime under its generic activation-traffic approximation.

```python
from mlsysim.engine.engine import Engine
from mlsysim.hardware.registry import Hardware
from mlsysim.models.registry import Models

model = Models.Language.Llama3_8B
hw = Hardware.Cloud.H100

for b in [1, 16, 128, 512]:
    perf = Engine.solve(model, hw, batch_size=b, precision="fp16")
    print(f"Batch {b}: {perf.bottleneck}-bound, MFU={perf.mfu:.3f}")
```

A.2 Volume 2 Style: Distributed Training

In Volume 2, the focus shifts to distributed systems and fleet-level economics. LEGO-style cells here compose multiple solvers to estimate the time and cost of training frontier models across thousands of GPUs:

Listing 6: **Illustrative LEGO-Style Cell: Training Economics.** Composing solvers to estimate the duration and carbon footprint of a large-scale training run.

```python
from mlsysim.solvers import DistributedModel, SustainabilityModel
from mlsysim.systems.registry import Systems
from mlsysim.models.registry import Models
from mlsysim.infrastructure.registry import Infrastructure

fleet = Systems.Clusters.Frontier_8K
model = Models.Language.Llama3_70B

perf = DistributedModel().solve(
    model, fleet, batch_size=4096, seq_len=4096, tp_size=8, pp_size=8
)
training_days = (perf.step_latency_total * 1e12 / (4096 * 4096)).to("day")

cluster_mfu = perf.scaling_efficiency * perf.node_profile.mfu
sust = SustainabilityModel().solve(
    fleet, duration_days=training_days.magnitude,
    datacenter=Infrastructure.Datacenters.Iowa_Reference, mfu=cluster_mfu
)
print(f"Training Time: {training_days.magnitude:.0f} days")
print(f"Carbon: {sust.carbon_footprint_kg / 1000:.0f} tonnes")
```

A.3 Volume 1 Style: Model Serving

Another common pattern in Volume 1 evaluates inference systems. Here, a LEGO-style cell uses the `ServingModel` to calculate the inter-token latency (ITL) for Llama-3 8B on an NVIDIA A100 GPU:

Listing 7: **Illustrative LEGO-Style Cell: Model Serving.** The `ServingModel` estimates the theoretical inter-token latency for an autoregressive generation step. The cell shows the four Load, Execute, Guard, and Output phases of the LEGO pattern.

```python
from mlsysim.solvers import ServingModel
from mlsysim.models.registry import Models
from mlsysim.hardware.registry import Hardware

# 1. LOAD (Constants)
model = Models.Language.Llama3_8B
hw = Hardware.Cloud.A100

# 2. EXECUTE (The Compute)
perf = ServingModel().solve(
    model, hw, seq_len=1256, batch_size=1, precision="int4", efficiency=0.5
)

# 3. GUARD (Invariants)
assert perf.feasible, "configuration must fit in memory"

# 4. OUTPUT (Formatting)
print(f"Inter-Token Latency (ITL): {perf.itl.to('ms'):.2f}")
```

Appendix B Registry Snapshot

The MLSys Zoo comprises 8 core registries spanning the full systems stack, plus the Agents, Embodied, Sensors, and Actuators extension registries and the `Literature` and `ReferenceStats` auxiliary surfaces. This appendix is a representative snapshot of the current local registry surface: enough to show the kinds of systems MLSYS·IM can compose, without treating the paper as the authoritative inventory. The typed Python and YAML registry source remains the source of truth for the complete, up-to-date entries.

B.1 Hardware Registry

Table 7 summarizes the current accelerator coverage by deployment tier. Each entry is a fully typed `HardwareNode` with `pint`-dimensioned fields. Users add custom entries by instantiating the same types (Listing 2), and custom entries participate in the same dimensional pipeline as vetted ones. For example, a research team evaluating a pre-release accelerator can define a speculative entry with projected specifications and immediately sweep it against existing workloads (Listing 8).

Listing 8: **Extending the Registry.** Defining a speculative hardware entry using the typed `HardwareNode` schema (Table 9) for immediate solver evaluation.

```python
from mlsysim.hardware.types import HardwareNode, ComputeCore, MemoryHierarchy
from mlsysim.core.units import TFLOPs, GB, TB, second, watt

custom_accelerator = HardwareNode(
    name="Speculative GPU 2027", release_year=2027,
    compute=ComputeCore(peak_flops=1200 * TFLOPs / second,
                        precision_flops={"fp16": 1200 * TFLOPs / second, "fp8": 2400 * TFLOPs / second}),
    memory=MemoryHierarchy(capacity=144 * GB, bandwidth=5.0 * TB / second),
    tdp=800 * watt,
)
```

Table 7: **Hardware Registry Snapshot.** All 52 hardware entries in the registry, organized by deployment tier and sorted by peak throughput. Peak FLOP/s is FP16 Tensor Core (or equivalent) throughput; BW is primary-memory bandwidth (HBM, LPDDR, or on-die SRAM as appropriate); memory is binary capacity in GiB.

Tier	Accelerator	Peak (TFLOP/s)	Mem (GiB)	BW (TB/s)	TDP (W)	Interconnect
Cloud						
	Intel SGX Enclave (Reference)	1	0.125	0.01	–	—
	Reference Desktop CPU	1	64	0.05	150	—
	NVIDIA Tesla K80 (one GK210 GPU)	4.37	12	0.24	150	—
	NVIDIA Tesla P100	21.2	16	0.732	300	—
	Google TPU v2	46	16	0.7	280	—
	NVIDIA T4	65	16	0.32	70	—
	Google TPU v1	92	8	0.034	75	—
	NVIDIA V100 PCIe	112	32	0.9	250	PCIe Gen3 x16
	Google TPU v3	123	32	0.9	450	ICI (280 GB/s)
	NVIDIA V100	125	32	0.9	300	NVLink 2.0 (300 GB/s)
	Groq LPU	188	0.225	80	215	—
	Graphcore Colossus Mk2 GC200 IPU	250	0.879	45	–	—
	Google TPU v4	275	32	1.2	200	ICI (300 GB/s)
	NVIDIA A100	312	80	2.04	400	NVLink 3.0 (600 GB/s)
	NVIDIA A100 40GB	312	40	1.56	400	NVLink 3.0 (600 GB/s)
	AWS Trainium 2	380	96	2.4	500	—
	AMD MI250X	383	128	3.2	500	—
	Intel Gaudi 2	432	96	2.45	600	RoCE (24x100GbE) (300 GB/s)
	Google TPU v5p	459	95	2.76	300	ICI (1,200 GB/s)
	NVIDIA H100 PCIe	756	80	2	350	—
	Google TPU v6e (Trillium)	918	32	1.64	300	ICI (800 GB/s)
	NVIDIA H100	989	80	3.35	700	NVLink 4.0 (900 GB/s)
	NVIDIA H200	989	141	4.8	700	NVLink 4.0 (900 GB/s)
	AMD MI300X	1,307	192	5.3	750	—
	AMD Instinct MI325X	1,307	256	6	1,000	—
	Intel Gaudi 3	1,835	128	3.7	900	—
	Untether AI speedAI240	2,000	0.232	1,000	–	—
	NVIDIA B200	2,250	180	8	1,000	NVLink 5.0 (1,800 GB/s)
	d-Matrix Corsair	2,400	2	150	600	—
	Cerebras CS-3 (WSE-3)	125,000	44	21,000	23,000	SwarmX / MemoryX
	NVIDIA GB200 NVL72	162,000	13,824	576	120,000	NVLink Switch (Bisection)
Workstation						
	NVIDIA GeForce GTX 580 (3 GB)	1.58	3	0.192	244	PCIe Gen2 x16
	MacBook Pro (M3 Max)	14.2	128	0.4	100	—
	NVIDIA DGX Spark (GB10)	125	128	0.273	200	—
Edge						
	Edge Server	1	128	0.1	300	—
	Intel NUC + Movidius	1	16	0.025	15	—
	Google Coral Edge TPU	4	1	0.008	2	—
	RoboTaxi Reference Compute (NVIDIA DRIVE AGX Orin class)	5.2	32	0.2	60	Automotive sensor and Ethernet I/O
	NVIDIA Jetson Orin Nano	10	8	0.068	15	—
	NVIDIA Jetson Orin NX	25	16	0.102	25	—
	NVIDIA Jetson AGX Orin	43	64	0.204	60	—
	NVIDIA DRIVE Thor	1,000	64	0.546	100	Automotive 100G Ethernet and PCIe Gen5
Mobile						
	Google Pixel 8 (Tensor G3)	15	8	0.06	5	—
	Apple M2 Neural Engine	15.8	16	0.1	20	—
	iPhone 15 Pro (A17 Pro)	35	8	0.0512	5	—
	Snapdragon 8 Gen 3	45	12	0.077	5	—
Tiny						
	Nordic nRF52840 (Cortex-M4F)	0.000064	0.000977	0.000064	0.015	—
	Oura Ring 4 (wearable reference profile)	0.0001	0.00195	0.00008	0.02	—
	STM32F407 (Cortex-M4F)	0.000168	0.000183	0.000672	–	—
	Himax WE-I Plus	0.0002	0.00195	0.0001	0.005	—
	ESP32-S3 (AI)	0.0005	0.00391	0.00008	0.4	—
	Lockstep safety MCU (reference class)	0.0008	0.00391	0.0016	1	—

Notes: Cerebras CS-3 BW is on-wafer SRAM bandwidth; the weight-streaming serving case study (Section 7) instead uses the 1.2 TB/s MemoryX injection bandwidth seen by the host. NVIDIA GB200 NVL72 is a rack-scale system entry (72 Blackwell GPUs behind one NVLink switch domain) rather than a single accelerator.

B.2 Infrastructure and Systems Registries

The Hardware registry (Table 7) captures individual accelerator capabilities, but real deployments compose accelerators into nodes, connect nodes via network fabrics, and situate clusters in datacenters whose geography determines energy cost, carbon intensity, and cooling. The Infrastructure and Systems registries complete this picture: Infrastructure encodes *where* the fleet runs, and Systems encodes *how* accelerators form distributed configurations. Together the three registries let a single resolver chain evaluate end-to-end metrics from per-device roofline through fleet-level TCO and carbon footprint.

The Infrastructure registry encodes regional environmental context for sustainability analysis, plus datacenter profiles, rack power envelopes, cooling, cloud pricing, and capacity lead times. Carbon intensity varies by roughly 82× across the 12 registered grid profiles, making geography a first-order design variable (Case I2, Section 7). The Systems registry defines compute nodes, network fabrics, fleet configurations, TPU pods, switch fabrics, and storage and checkpoint profiles (NVMe, SATA, HDD, object stores, parallel file systems) used by `DistributedModel`, `ReliabilityModel`, and `DataModel`. Table 8 presents representative entries from both registries.

B.3 Type System and Physics API

To support the separation of demand and supply, MLSYS·IM relies on a shallow but rigorously typed hierarchy of Pydantic models (Table 9). These types act as the contract between the declarative registries and the analytical solvers. Because all physical quantities (bandwidth, latency, power) are dimensioned using the `pint` library, users can extend any registry by instantiating these types directly without risking unit mismatch errors.

Beneath the resolvers lies the Physics Engine, which contains the dimensioned mathematical identities that govern ML systems. Table 10 catalogs the public API of these physics modules. Each function is stateless, accepting dimensioned quantities (or raw floats that are immediately coerced to units) and returning dimensioned bounds. These functions implement the core logic for the Iron Law of training, the Roofline model, and network communication topologies.

Table 8: **Infrastructure and Systems Registry Snapshot.** *Top:* Grid profiles for sustainability analysis (CI = carbon intensity in gCO_2/kWh; PUE = power usage effectiveness; WUE = water usage effectiveness in L/kWh). *Bottom:* Registered compute nodes, network fabrics, and fleet configurations for distributed analysis.

Grid Profiles (12 registered)

Profile	CI	PUE	WUE	Source
Norway (Hydro)	10	1.06	0.0	Hydro
Quebec (Hydro)	20	1.06	0.0	Hydro
Iceland (Geothermal)	28	1.06	0.0	Geothermal
France (Nuclear)	50	1.12	1.8	Nuclear
EU Average	270	1.12	1.8	Mixed
Germany (Coal+Wind)	385	1.12	1.8	Mixed
Texas (ERCOT)	400	1.12	1.8	Mixed
US Average	429	1.12	1.8	Mixed
China Average	555	1.40	1.8	Coal
Iowa (Coal/Gas Reference)	680	1.12	1.8	Coal/gas
India Average	720	1.58	1.8	Coal
Poland (Coal)	820	1.58	1.8	Coal

Nodes, Fabrics, and Fleets

Compute Nodes (8 registered)	
DGX A100	8× NVIDIA A100, NVLink 3.0, 300 GB/s per direction
DGX B200	8× NVIDIA B200, NVLink 5.0, 900 GB/s per direction
DGX H100	8× NVIDIA H100, NVLink 4.0, 450 GB/s per direction
Meta Grand Teton (8x H100)	8× NVIDIA H100, NVLink 4.0, 450 GB/s per direction
HGX H100 (Dual AMD EPYC 9654)	8× NVIDIA H100, NVLink 4.0, 450 GB/s per direction
Kempner H100 4-GPU Server	4× NVIDIA H100, NVLink 4.0, 450 GB/s per direction
8x MI300X Server	8× AMD MI300X, intra-node link, 896 GB/s per direction
MacBook Pro M3 Max Workstation	1× MacBook Pro (M3 Max), intra-node link, 400 GB/s per direction
Network Fabrics (14 registered)	
10GbE	10 Gb/s, 50 μs
100GbE	100 Gb/s, 50 μs
100GbE RoCE	100 Gb/s, 10 μs
IB EDR	100 Gb/s
IB HDR	200 Gb/s, 7 μs
400GbE	400 Gb/s, 50 μs
400GbE RoCE	400 Gb/s, 10 μs
IB NDR	400 Gb/s, 5 μs
IB NDR (2:1 oversubscribed)	400 Gb/s, 5 μs
IB NDR (4:1 oversubscribed)	400 Gb/s, 5 μs
800GbE	800 Gb/s, 50 μs
IB XDR	800 Gb/s, 5 μs
1.6TbE	1,600 Gb/s, 50 μs
IB GXDR	1,600 Gb/s, 5 μs
Fleet Configurations (20 registered)	
Lab Cluster (64 A100 GPUs)	8× DGX A100, IB HDR
Lab Cluster (64 H100 GPUs)	8× DGX H100, IB HDR
DGX SuperPOD scalable unit (32 DGX H100)	32× DGX H100, IB NDR
Research Cluster (256 GPUs)	32× DGX H100, 100GbE
Kempner AI Cluster H100 Partition (384 H100 GPUs)	96× Kempner H100 4-GPU Server, IB NDR
Training Cluster (512 H100 GPUs)	64× DGX H100, IB HDR
Training Cluster (1024 A100 GPUs)	128× DGX A100, IB HDR
Training Cluster (1024 B200 GPUs)	128× DGX B200, IB XDR
Training Cluster (1024 GPUs)	128× DGX H100, IB HDR
Training Cluster (1024 GPUs, NDR InfiniBand)	128× DGX H100, IB NDR
Production Cluster (2048 GPUs)	256× DGX H100, IB HDR
Production Cluster (2048 GPUs, NDR InfiniBand)	256× DGX H100, IB NDR
Frontier Cluster (8192 GPUs)	1,024× DGX H100, IB NDR
Training Cluster (10000 GPUs)	1,250× DGX H100, IB NDR
Meta AI Research SuperCluster (16,000 A100)	2,000× DGX A100, IB HDR
Meta Llama 3 cluster (16,384 H100)	2,048× DGX H100, IB NDR
Meta GenAI cluster (24,576 H100, InfiniBand)	3,072× Meta Grand Teton (8x H100), IB NDR
Meta GenAI cluster (24,576 H100, RoCE)	3,072× Meta Grand Teton (8x H100), 400GbE RoCE
Reference H100 Cluster (25000 GPUs)	3,125× DGX H100, IB NDR
Mega Cluster (100000 GPUs)	12,500× DGX H100, IB NDR

Table 9: **Core Type Schema.** Pydantic `BaseModel` types that compose registry entries. The hierarchy is deliberately shallow: `HardwareNode` aggregates `ComputeCore` and `MemoryHierarchy` as direct fields, not through deep inheritance. All physical fields are `pint Quantity` objects. Users extend any registry by instantiating these types directly (Listing 2).

Type	Key Fields
`ComputeCore`	`peak_flops` (FLOP/s), `precision_flops` (dict: tf32/fp8/int8/fp32 → FLOP/s; FP16 is the base `peak_flops`), `sm_count`
`MemoryHierarchy`	`capacity` (bytes), `bandwidth` (bytes/s); optional SRAM, flash, and L2 tiers
`IOInterconnect`	`bandwidth` (bytes/s), `latency` (seconds), `name`
`HardwareNode`	`compute`, `memory`, `storage`, `interconnect`, `nvlink`, `tdp` (W), `unit_cost` (USD), `dispatch_tax` (s), `metadata.provenance`
`ComputationGraph`	`total_ops` (FLOPs), `weight_bytes` (bytes), `arithmetic_intensity` (FLOP/byte), `parameter_count`, `layers`
`TransformerWorkload`	`name`, `architecture`, `parameters`, `layers`, `hidden_dim`, `heads`, `kv_heads`
`GridProfile`	`carbon_intensity_g_kwh`, `pue`, `wue` (L/kWh), `renewable_pct`
`NetworkFabric`	`topology`, `bandwidth` (bytes/s), `latency` (s), `oversubscription_ratio`
`Fleet`	`node`, `count`, `fabric` (NetworkFabric), `region/datacenter`, `mtbf_hours`
`SystemEvaluation`	`feasibility`, `performance`, `macro` (three-level scorecard)

Table 10: **Physics Module Public API.** Selected function signatures organized by domain module. Every function accepts `pint Quantity` arguments (or raw floats coerced via `_ensure_unit()`) and returns dimensioned quantities. Import from `mlsysim.physics` (one module per domain).

Module	Function	Signature (key parameters → return)
performance	dTime	(total_ops, n_devices, peak_flops, η) → seconds
	calc_bottleneck	(ops, model_bytes, device_flops, device_bw) → {time, regime}
	calc_amdahls_speedup	(parallel_frac, speedup_factor) → overall speedup
	calc_pipeline_bubble	(n_stages, n_microbatches, v_stages) → bubble fraction
	calc_effective_flops	(peak_flops, mfu, scaling_eff, goodput_ratio) → sustained FLOP/s
	calc_training_time_days	(total_ops, n_devices, peak_flops, η) → days
memory	model_memory	(params, bytes_per_param, unit) → memory
	calc_activation_memory	(n_layers, seq_len, batch_size, hidden_dim, n_heads, precision_bytes, strategy) → bytes
	calc_kv_cache_size	(layers, heads, dim, seq, batch, prec) → bytes
	calc_paged_kv_cache_size	(same + page_size) → (bytes, last-page waste %)
	calc_mla_cache_size	(n_layers, kv_lora_rank, qk_rope_head_dim, seq, batch, prec) → bytes
	calc_checkpoint_size	(n_params, bytes_per_param) → bytes
serving	calc_queue_latency_mmc	(arrival_rate, service_rate, n_servers) → wait time
communication	calc_ring_allreduce_time	(msg_bytes, n_gpus, bw, latency) → seconds
	calc_tree_allreduce_time	(msg_bytes, n_gpus, bw, latency) → seconds
	calc_all_to_all_time	(msg_bytes, n_gpus, bw, latency) → seconds
	calc_hierarchical_allreduce_time	(message_bytes, n_nodes, gpus_per_node, intra_node_bw, inter_node_bw, intra_node_lat, inter_node_lat) → seconds
reliability	calc_young_daly_interval	(ckpt_cost, mtbf) → optimal interval
	calc_mtbf_cluster	(component_mtbf, n, correlation) → cluster MTBF
	calc_mtbf_node	(GPU, NIC, PSU, and other component MTBFs with counts) → node MTBF
	calc_availability_stacked	(single_avail, n_replicas) → stacked availability
	calc_failure_probability	(mtbf, duration) → probability
transformer	calc_transformer_training_flops	(n_params, n_tokens) → total FLOPs
	calc_transformer_decode_flops	(n_params, n_tokens) → decode FLOPs
economics	calc_fleet_tco	(unit_cost, power, quantity, years, kwh_price) → TCO
	calc_monthly_egress_cost	(bytes_per_sec, cost_per_gb) → monthly cost
networking	calc_network_latency_ms	(distance_km) → latency
statistics	calc_population_stability_index	(expected, actual) → PSI
	calc_two_proportion_sample_size	(baseline, lift, z_α, z_β) → n
	calc_constraint_propagation_factor	(from, to, base) → factor
agents	calc_trajectory_reliability	(step_reliabilities) → trajectory success prob
	calc_test_time_compute_cost	(base_cost, k_samples, verify_cost) → total cost
	calc_multiagent_optimal_concurrency	(work_tokens, comm_overhead, n_workers) → optimal workers
robotics	calc_sensor_to_actuator_latency	(sensor_ms, compute_ms, bus_ms, actuator_ms) → total latency
	calc_safe_stopping_distance	(velocity, latency, max_decel) → stopping distance
	calc_action_chunk_cadence	(chunk_size, control_freq, policy_latency) → (cadence, margin)
drivetrain	calc_traction_deceleration_limit	(friction_coefficient, drive_load_share) → deceleration ceiling
	calc_tipover_deceleration_limit	(cg_height, tip_arm) → deceleration ceiling
	calc_drive_torque_deceleration_limit	(wheel_torque_max, wheel_radius, n_driven_wheels, mass) → deceleration ceiling
	calc_braking_limits	(mass, deceleration, drive and geometry parameters) → {ceilings, binding, passes}
quantities	transfer_time / compute_time	(payload, bandwidth) / (work, throughput) → time
	energy_from_power / carbon_from_energy	(power, duration) / (energy, CI) → energy / mass
	memory_from_params / token_throughput	(params, bytes/param) / (tokens, duration) → bytes / rate

B.4 Code Examples Cookbook

To demonstrate how the theoretical 3-tier architecture from Section 5 is exposed in Python, we provide eight complete, runnable examples. Each example is written as a small engineering task: the prose names the question, the listing shows which registry facts are selected, and the returned object exposes the bound, bottleneck, or scorecard used to make a decision. The point is not to hide the model behind an API call, but to show how registry-backed composition turns a systems question into a repeatable calculation.

B.4.1 Scenario A: Rapid Parametric Sweeps (Tier 1)

Scenario: A batch-size sweep shows how weight reads amortize under the generic engine's activation-traffic approximation. The code selects a workload and hardware entry, then calls `Engine.solve()` once per batch size. Each result reports throughput, MFU, and the binding Roofline regime. This example does not model continuous batching or KV-cache traffic; those require `ServingModel` or `ContinuousBatchingModel`.

Listing 9: **Programmatic Sweeps (Tier 1).** Using the `Engine` to sweep a workload across a hardware constraint space (batch size). Throughput rises from 178 to 2,002 tokens/s and then saturates: per-request memory traffic grows with the batch, the generic activation-traffic approximation remains memory-bound across this sweep. Section 6.2 reports local sweep runtime.

```python
from mlsysim.engine.engine import Engine
from mlsysim.hardware.registry import Hardware
from mlsysim.models.registry import Models
import pandas as pd

# Sweep Llama 3 8B across multiple batch sizes on an H100 GPU
model = Models.Language.Llama3_8B
hardware = Hardware.Cloud.H100

results = []
for batch_size in [1, 8, 32, 128, 256]:
    # Engine.solve() wraps the Tier 1 SingleNodeModel
    perf = Engine.solve(model, hardware, batch_size=batch_size, precision="fp16")

    results.append({
        "Batch Size": batch_size,
        "Throughput (tok/s)": round(perf.throughput.magnitude),
        "Bottleneck": perf.bottleneck,
        "MFU": round(perf.mfu, 3)
    })

df = pd.DataFrame(results)
print(df)
#    Batch Size  Throughput (tok/s) Bottleneck    MFU
# 0   1   178   Memory   0.003
# 1   8   893   Memory   0.014
# 2   32   1,564   Memory   0.025
# 3   128   1,925   Memory   0.031
# 4   256   2,002   Memory   0.033
```

B.4.2 Scenario B: Cross-Domain Carbon Accounting (Tier 1 Composition)

Scenario: You have been tasked with estimating the carbon footprint of training a 70B parameter model. The listing first uses `DistributedModel` to compute the step latency for a registry-backed 256-GPU fleet, then converts a one-trillion-token target into a training duration. That duration becomes the input to `SustainabilityModel`, where the Iowa datacenter profile supplies grid intensity and facility overhead; the composition works because solver outputs are dimensioned quantities rather than untyped floats.

Listing 10: **Resolver Composition (Tier 1).** Output quantities from one solver (the computed step latency from the `DistributedModel`) feed directly into subsequent solvers (the `SustainabilityModel`), enabling end-to-end full-stack analysis without manual unit conversion.

```python
from mlsysim.solvers import DistributedModel, SustainabilityModel
from mlsysim.systems.registry import Systems
from mlsysim.infrastructure.registry import Infrastructure
from mlsysim.models.registry import Models

fleet = Systems.Clusters.Research_256  # 32x DGX H100 nodes

# 1. Performance Domain: how long does one training step take?
perf = DistributedModel().solve(
    model=Models.Language.Llama3_70B, fleet=fleet,
    batch_size=1024, seq_len=2048, tp_size=8, pp_size=1,
    activation_recomputation=True,
)

# 2. Train for 1T tokens: convert step latency into a duration
steps = 1e12 / (1024 * 2048)
training_days = (perf.step_latency_total * steps).to("day").magnitude

# 3. Sustainability Domain: what is the carbon impact?
cluster_mfu = perf.scaling_efficiency * perf.node_profile.mfu
sust = SustainabilityModel().solve(
    fleet=fleet, duration_days=training_days,
    datacenter=Infrastructure.Datacenters.Iowa_Reference,  # high-carbon grid
    mfu=cluster_mfu,
)
print(f"Training Duration: {training_days:.0f} days")
# Training Duration:  74 days
print(f"Total Carbon: {sust.carbon_footprint_kg / 1000:.1f} tonnes CO2e")
# Total Carbon:  117.3 tonnes CO2e
```

B.4.3 Scenario C: Uncovering the Data Wall (Tier 1)

Scenario: Your GPU utilization is mysteriously low during a computer vision training job. The code splits the diagnosis into two checks: `DataModel` compares the required image stream against the node's I/O path, while `TransformationModel` compares CPU preprocessing throughput against accelerator step time. Reading the two results together distinguishes a storage or transfer bottleneck from a host-side decode and augmentation bottleneck.

Listing 11: **Data Pipeline Analysis (Tier 1).** Evaluating the "Data Wall". This example separates I/O bandwidth bottlenecks (storage to GPU) from CPU transformation bottlenecks (decoding and augmenting data), in the spirit of Case Study S2 but with an under-provisioned 16-worker pipeline.

```python
from mlsysim.solvers import DataModel, TransformationModel
from mlsysim.hardware.registry import Hardware
from mlsysim.core.units import Q_

# 1. Define the ingestion demand (e.g., training a vision model)
# We need to feed 40,000 images per second to the GPU.
demand_rate = Q_("40000 1/s") * Q_("150 KB")  # ~6 GB/s

# 2. Check if the A100 node's I/O path can handle the bandwidth
data_result = DataModel().solve(
    workload_data_rate=demand_rate,
    hardware=Hardware.Cloud.A100
)
print(f"I/O Stalled: {data_result.is_stalled}")  # False

# 3. Check if 16 CPU workers can decode + augment fast enough
#    (850 images/s per worker, 150 KB per image)
cpu_throughput = 16 * 850 * Q_("150 KB/s")
transform_result = TransformationModel().solve(
    batch_size=2048,
    sample_size_bytes=Q_("150 KB"),
    cpu_throughput=cpu_throughput,
    accelerator_step_time=Q_("48 ms"),
)
print(f"CPU Stalled: {transform_result.is_bottleneck}")  # True
print(f"Accel. Utilization: "
      f"{transform_result.accelerator_utilization:.0%}")  # 32%
```

B.4.4 Scenario D: SLA-Driven Hardware Synthesis (Tier 2)

Scenario: You are provisioning hardware for a new LLM application. You do not start with a GPU SKU; you start with the business requirement that the application should emit a token every 30 milliseconds. The `SynthesisSolver` inverts the same first-order Roofline model used for evaluation: given `Models.Language.Llama3_8B`, the target latency, and precision, it returns the memory-bandwidth and compute-throughput requirements that candidate hardware must clear.

Listing 12: **SLA-Driven Hardware Synthesis (Tier 2).** Instead of evaluating a known piece of hardware, the `SynthesisSolver` works backward. Given a strict latency SLA, it algebraically inverts the Roofline equations to output first-order minimum memory bandwidth and compute throughput requirements for procurement screening.

```python
from mlsysim.solvers import SynthesisSolver
from mlsysim.models.registry import Models
from mlsysim.core.units import Q_

# We want to serve Llama-3 8B.
# Our strict SLA dictates an inter-token latency of 30 ms.
# What is the minimum hardware capability required to achieve this?

solver = SynthesisSolver()
requirements = solver.solve(
    model=Models.Language.Llama3_8B,
    target_latency=Q_("30 ms"),
    precision="fp16"
)

# Output the hardware requirements
print(f"Required HBM Bandwidth: {requirements.required_bw.to('GB/s'):.1f}")
# Required HBM Bandwidth:  535.3 GB / second
print(f"Required Compute:       {requirements.required_flops.to('TFLOPs/s'):.1f}")
# Required Compute:  1.1 TFLOPs / second
```

B.4.5 Scenario E: Speculative Decoding Speedup (Tier 1)

Scenario: You are serving a large frontier model and want to know how much faster inference would be if you used a smaller, cheaper draft model to speculatively decode tokens. The first call establishes the autoregressive baseline for the target model on a single B200, which holds the FP16 Llama-3 70B weights that an 80 GB H100 cannot. The second call adds a draft model and an acceptance-rate assumption, allowing `ServingModel` to account for draft generation and target-model verification before reporting the effective inter-token latency and speedup.

Listing 13: **Speculative Decoding Analysis (Tier 1).** The `ServingModel` natively supports algorithmic optimizations like speculative decoding. By providing a draft model and an expected acceptance rate, the solver calculates the effective inter-token latency (ITL) by comparing compute and memory costs for each phase and including the bonus-token target query.

```python
from mlsysim.solvers import ServingModel
from mlsysim.hardware.registry import Hardware
from mlsysim.models.registry import Models

# Serve Llama-3 70B using Llama-3 8B as a draft model
target_model = Models.Language.Llama3_70B
draft_model = Models.Language.Llama3_8B
hardware = Hardware.Cloud.B200

solver = ServingModel()

# Standard Autoregressive Decoding
base_result = solver.solve(target_model, hardware, seq_len=2048, batch_size=1)
print(f"Standard ITL: {base_result.itl.to('ms'):.2f}")
# Standard ITL: 18.53 millisecond

# Speculative Decoding
spec_result = solver.solve(
    target_model, hardware, seq_len=2048, batch_size=1,
    draft_model=draft_model,
    draft_acceptance_rate=0.75
)
print(f"Speculative ITL: {spec_result.itl.to('ms'):.2f}")
# Speculative ITL: 9.17 millisecond
print(f"Speedup: {(base_result.itl / spec_result.itl).magnitude:.2f}x")
# Speedup:  2.02x
```

B.4.6 Scenario F: The Full-Stack Scorecard (Tier 2)

Scenario: Rather than invoking individual solvers, you want the single "will it run, is it fast enough, and what does it cost" verdict for a deployment. An executable `Scenario` is a named bundle of workload, target system, SLA, and operating context. Calling `evaluate()` computes the Feasibility, Performance, and Macro levels and returns a `SystemEvaluation` whose scorecard preserves the reason for each pass or failure; `validate()` instead raises at the first failing level.

Listing 14: **Three-Level Scorecard (Tier 2).** `Scenario.evaluate()` composes the resolver chain automatically and returns a `SystemEvaluation`: Level 1 checks memory feasibility, Level 2 reports latency against the SLA, and Level 3 reports cost and carbon.

```python
import mlsysim

# A pre-built executable scenario: chatbot serving with an SLA + grid context
evaluation = mlsysim.Scenarios.ChatbotServing.evaluate()
print(evaluation.scorecard())
# (abridged)
# Level 1:  Feasibility [PASS] Model fits in memory (16.1 GB / 85.9 GB)
# Level 2:  Performance [PASS] Latency:  5.60 ms (Target:  500 ms)
# Level 3:  Macro/Economics [PASS] Annual Carbon:  2946.3 kg | TCO: $10,407
print(evaluation.passed_all)  # True
```

B.4.7 Scenario G: Automated Parallelism Search (Tier 3)

Scenario: You have a fixed cluster and a large model and want the framework to *find* the best 3D-parallelism split rather than specifying it by hand. The Tier 3 `ParallelismOptimizer` enumerates power-of-two TP $\times$ PP $\times$ DP factorizations of the available devices, computes per-GPU training state for each candidate, and discards layouts that exceed HBM. The result keeps the best MFU configuration and a ranked candidate list, making the search procedure visible rather than a black-box recommendation.

Listing 15: **Parallelism Optimizer (Tier 3).** The optimizer enumerates valid TP/PP/DP factorizations of the cluster, discards configurations whose per-GPU training state exceeds HBM, and returns the highest-MFU survivor with its ranked candidate list.

```python
import mlsysim
from mlsysim.solvers import ParallelismOptimizer

fleet = mlsysim.Systems.Clusters.Research_256   # 256x H100
result = ParallelismOptimizer().solve(
    mlsysim.Models.Language.Llama3_70B, fleet,
    batch_size=2048, precision="fp16", efficiency=0.5,
)
best = result.best_config
print(f"TP={best['tp']}, PP={best['pp']}, DP={best['dp']}")
# TP=2, PP=16, DP=8
print(f"MFU: {result.best_mfu:.3f}")  # MFU: 0.458
for c in result.top_candidates[1:3]:
    k = c["config"]
    print(f"TP={k['tp']}, PP={k['pp']}, DP={k['dp']} {c['mfu']:.3f}")
# TP=2, PP=32, DP=4 0.456
# TP=2, PP=128, DP=1 0.452
```

B.4.8 Scenario H: From Datacenter to Microcontroller (Tier 1)

Scenario: The same dimensionally strict engine that sizes an H100 fleet also sizes a microcontroller. The loop changes only the registry-backed model and hardware entries; `Engine.solve()` still lowers the workload demand, reads the supply constraints, and returns feasibility, bottleneck regime, latency, and per-inference energy. This is the benefit of the demand–supply contract: datacenter and TinyML examples share the same code path while drawing facts from very different parts of the registry.

Listing 16: **Edge and TinyML Deployment (Tier 1).** The identical `Engine.solve` call evaluates milliwatt-class devices: a LeNet variant fits in the ESP32-S3's on-chip SRAM, while MobileNet variants run on a Jetson Orin Nano and a Coral Edge TPU. Energy is per inference.

```python
from mlsysim.engine.engine import Engine
from mlsysim.hardware.registry import Hardware
from mlsysim.models.registry import Models

deployments = [
    (Models.Vision.LeNet1,               Hardware.Tiny.ESP32_S3),
    (Models.Vision.MobileNetV2,          Hardware.Edge.JetsonOrinNano),
    (Models.Vision.MobileNetV2_Alpha0_5, Hardware.Edge.Coral),
]
for model, hw in deployments:
    perf = Engine.solve(model, hw, batch_size=1, precision="int8")
    print(f"{hw.name:24s} {perf.bottleneck}-bound  "
          f"{perf.latency.to('ms'):.2f}  {perf.energy.to('mJ'):.2f}")
# ESP32-S3 (AI) Compute-bound 1.03 millisecond 0.13 millijoule
# NVIDIA Jetson Orin Nano Memory-bound 0.80 millisecond 3.74 millijoule
# Google Coral Edge TPU Memory-bound 1.28 millisecond 0.77 millijoule
```